POLI TÉCNICO GUARDA

Escola Superior de Tecnologia e Gestão

CYBERSECURITY IN NETWORKS AND INTRUSION DETECTION SYSTEMS

INTERNSHIP REPORT
FOR THE DIPLOMA FOR PROFESSIONAL TECHNICIANS IN CYBERSECURITY

Pablo María Basanta Domínguez
June / 2024

POLI TÉCNICO GUARDA

Escola Superior de Tecnologia e Gestão

CYBERSECURITY IN NETWORKS AND INTRUSION DETECTION SYSTEMS

INTERNSHIP REPORT
FOR THE DIPLOMA FORPROFESSIONAL TECHNICIANS IN CIBERSECURITY

Professor Orientador: Prof. Noémio de Jesus da Encarnação Dória

1707282
Pablo María Basanta Domínguez

June / 2024

Abstract

During my Guarda Polytechnic University (IPG) curricular internship at OneSource, I was assigned a set of activities centred on research and development of advanced solutions in the field of network cybersecurity and intrusion detection systems. To carry out these activities, I used the programming language Python.

During this internship, my aim was to research and apply various technologies to improve cybersecurity and the integrity of intrusion detection systems. To carry out my activities, I researched and applied Federated Learning, which makes it possible to train the machine learning model in a distributed way without the problem of sharing sensitive data between nodes.

The implementation of differential privacy and zero-knowledge proof are also crucial parts of my project. These technologies not only allow calculations to be carried out on encrypted data, ensuring that sensitive information is never revealed in the process, but also provide a way of verifying that information is true without actually revealing the trafficker.

For container management and service orchestration, I went through all the phases using Docker and Kubernetes. They have helped me ensure that I can create consistent and scalable development and production environments.

Finally, I worked with Message Queuing Telemetry Transport (MQTT), a lightweight messaging protocol, primarily utilizing it for messaging purposes. The integration of MQTT has enabled the development of solutions that can work effectively in distributed networks and heterogeneous devices

Keywords:

Cybersecurity, Python, Supervised Learning, Unsupervised Learning, Federated Learning, Differential Privacy, Zero-knowledge Proof, Docker, Kubernetes, MQTT

Resumo

Durante o estágio curricular do IPG, realizado na OneSource, foi-me atribuído um conjunto de actividades centradas na investigação e desenvolvimento de soluções avançadas no domínio da cibersegurança das redes e dos sistemas de deteção de anomalias na rede. Para realizar estas atividades, utilizei a linguagem de programação Python.

Durante este estágio, o meu objetivo era investigar e aplicar várias tecnologias para melhorar a cibersegurança e a integridade do sistema de deteção de intrusões. Para a execução das minhas atividades, pesquisei e apliquei a Aprendizagem Federada, que permite treinar o modelo de aprendizagem automática de forma distribuída sem receber o problema da partilha de dados sensíveis entre nós.

A implementação da privacidade diferencial e a prova de conhecimento zero são também consideradas partes cruciais do meu projeto. Estas tecnologias não só permitem que os cálculos sejam efetuados em dados encriptados, garantindo que as informações sensíveis nunca serão reveladas no processo, como também fornecem uma forma de verificar se as informações são verdadeiras sem revelar efetivamente o traficante.

Para a gestão de contentores e orquestração de serviços, passei por todas as fases, utilizando Dockers e Kubernetes. Eles ajudaram-me a garantir que posso criar ambientes de desenvolvimento e produção consistentes e escaláveis.

Por último, trabalhei com o MQTT, um protocolo de mensagens leve, utilizando-o principalmente para fins de envio de mensagens. A integração do MQTT permitiu o desenvolvimento de soluções que podem funcionar eficazmente em redes distribuídas e dispositivos heterogéneos.

Palavras-chave:

Cibersegurança, Python, Aprendizagem Supervisionada, Aprendizajem não Supervisionada, Aprendizagem Federada, Privacidade Diferencial, Prova de Conhecimento Zero, Dockers, Kubernetes, MQTT.

Acknowledgments

To my love Sofia Saraiva, my greatest support in this journey towards a better future. I left my hometown and crossed borders to be by your side, and every step we have taken together. Your unconditional love and constant encouragement have been the driving force behind my achievements. This final report represents not only my dedication, but also our shared commitment to a future filled with opportunity. Thank you for being my rock, my motivation and my undying love. This achievement is as much yours as it is mine.

I would like to thank my family, my parents Andrés and Paloma, my sister María and my brother-in-law Diego, who despite the distance, are still there, helping and supporting me. Their advice and their teachings, which they pass on to me every day, have made this new challenge relatively easier with them far away but at the same time close.

My deepest thanks to my internship advisor, my professor and friend Noémio Dória, for all his help, support and guidance in my difficult moments, both professionally and personally. Thanks to your help and guidance this has been possible. You have shown that a teacher is a great influence, that you not only teach in the classroom, but that this extends beyond the IPG where I started my training as a Cybersecurity technician.

A big thank you to my friends and classmates, Tomás, Duarte, and Sylvain, for welcoming me and making me feel integrated as one of them despite the language differences. Although it was quite frustrating many times to have conversations because of my Portuguese and the resulting jokes, you made me realize that I was not alone. And a special thanks to my friend and flatmate, João, who despite the fact that neither of us speak Portuguese, we managed to have a good time and enjoy ourselves during this whole period.

We would also like to thank the Guarda Polytechnic University (IPG) and its professors for their dedication to teaching their knowledge about Cybersecurity. Special mention to Professor Fernando Melo, director of the Cybersecurity CTeSP. And also special mention to María del Carmen Arau Ribeiro, a teacher who always transmitted her positivity and her desire to learn with her enormous curiosity, a teacher full of projects that made her classes and learning English something wonderful.

Finally, I'd like to thank the company OneSource for the opportunity to learn and improve in an area such as machine learning, which has been a constant challenge throughout the internship period. I would especially like to thank my colleague Marco for welcoming me as a member of the team and making this adventure more bearable. I would also like to thank my manager Pedro Felix, for his help, his time, his patience and his advice. Thanks overall to the OneSource team.

Index

Abstract i

Resumo ii

Acknowledgments iii

Index 1

Index of Figures 3

List of acronyms and abbreviations 4

1 Introduction 5
1.1 Hosting Entity 5
1.2 Institution 6
1.3 Job Objectives 6
1.4 Structure of the Document 7

2 State of the Art 8
2.1 Introduction to Machine Learning 8
2.1.1 Definition and Context 8
2.1.2 Brief history and evolution 9
2.2 Types of Machine Learning 10
2.2.1 Supervised Learning 10
2.2.2 Unsupervised Learning 11
2.2.3 Reinforcement Learning 12
2.3 Deep Learning 14
2.3.1 Convolutional Neural Networks 14
2.3.2 Recurrent Neural Networks 15
2.3.3 Pre-trained Neural Network Architectures 15
2.3.4 Autoencoders 16
2.4 Challenges in Machine Learning 17
2.4.1 Overfitting 17
2.4.2 Regularization and Overfitting Reduction Techniques 17

2.5 Technologies for Privacy Enhancement . . . 18
2.5.1 Homomorphic Encryption . . . 18
2.5.2 Differential Privacy . . . 20
2.5.3 Zero-Knowledge Proofs . . . 21
2.5.4 Multi-Party Computation . . . 22
2.6 Federated Learning . . . 23
2.6.1 Definition and Basic Principles . . . 23
2.6.2 Applications and Use Cases . . . 24
2.6.3 Challenges and Security Considerations . . . 24
2.6.4 Advantages and Disadvantages of Federated Learning . . . 25

3 Development Tools . . . 27
3.1 Visual Studio Code . . . 27
3.2 Python . . . 27
3.3 TensorFlow . . . 28
3.4 Flower . . . 28
3.5 Docker . . . 29
3.6 Kubernetes . . . 30
3.7 Message Queuing Telemetry Transport . . . 31
3.7.1 Mosquitto . . . 32

4 Development Work . . . 34
4.1 Initial Part . . . 34
4.2 Implementation . . . 34
4.2.1 Client . . . 36
4.2.2 Server . . . 38
4.2.3 Collector . . . 38
4.2.4 Generator . . . 39
4.3 Results . . . 39
4.3.1 Training Perfeormance Without Security . . . 39
4.3.2 Impact of Differential Privacy . . . 40
4.3.3 Impact of Zero-Knowledge Proofs . . . 40
4.3.4 Comparative Analysis . . . 40

5 Conclusion . . . 41

Bibliographical References . . . 42

Appendix . . . 46

Index of Figures

1.1 Company Logo 5

2.1 Supervised Learning 10
2.2 Unsupervised Learning 12
2.3 Reinforcement Learning 13
2.4 Deep Learning 14
2.5 Autoencoders 16
2.6 Overfitting 17
2.7 Homomorphic Encryption 19
2.8 Differential Privacy 20
2.9 Zero-Knowledge Proof 21
2.10 Multi-Party Computation 22
2.11 Federated Learning 24

5.1 Dockerfile 46
5.2 Requirementsl 47
5.3 .yaml 48
5.4 main.py Client 49
5.5 ClassClient.py 50
5.6 utils.py Client 51
5.7 LoadProcData.py Client 52
5.8 main.py Server 53
5.9 SqLite.py Server 54
5.10 Collector 55
5.11 Plugin Collector 56
5.12 Generator 57

List of acronyms and abbreviations

IT Information Technology

IPG Guarda Polytechnic University

PETs Privacy Enhancing Technologies

MQTT Message Queuing Telemetry Transport

IoT Internet of Things

ML Machine Learning

HE Homomorphic Encryption

DP Differential Privacy

ZKP Zero-Knowledge Proofs

MPC Multi-Party Computation

LDA Linear Discriminant Analysis

QDA Quadratic Discriminant Analysis

PCA Principal Component Analysis

t-SNE t-Distributed Stochastic Neighbour Embedding

DQN Deep Q-Network

TRPO Trust Region Policy Optimization

PPO Proximal Policy Optimization

A2C Advantage Actor-Critic

A3C Asynchronous Advantage Actor-Critic

Dagger Dataset Aggregation

GAIL Generative Adversarial Imitation Learning

CNNs Convolutional Neural Networks

RNNs Recurrent Neural Networks

LSTM Long Short-Term Memory

GRU Gated Recurrent Units

FHE Fully Homomorphic Encryption

PHE Partially Homomorphic Encryption

SHE Somewhat Homomorphic Encryption

APIs Application Programming Interface

TPUs Tensor Processing Unit

TFX TensorFlow Extended

QoS Quality of Service

ACL Access Control List

SSL Secure Sockets Layer

TLS Transport Layer Security

1 Introduction

Cybersecurity is becoming a fundamental pillar in the digital age, where the interconnectedness of devices and systems creates an environment prone to threats and vulnerabilities. In this context, the Cybersecurity Technician curricular internship at OneSource provided an ideal scenario for the exploration and implementation of advanced digital protection strategies.

This report this work is based on was carried out within the scope of the curricular internship, which is integrated in the study plan of the 2nd semester of the 2nd year of the IPG degree in Cibersecurity.

1.1 Hosting Entity

The internship was carried out in the company OneSource, Consultoria Informática Lda. OneSource is an Information Technology (IT) company specialized in the fields of data communications, security, networks and systems management, including consulting, auditing, design, development and lifetime management of specialised IT solution for corporate networks, public sector institutions, utilities and telecommunications operators.

OneSource is based in Coimbra, Portugal.

ONESOURCE
Consultoria Informática Lda.

Figure 1.1: Company Logo

1.2 Institution

The curricular internship corresponds to the CTeSP in Cybersecurity, at the IPG. This institution of higher education, at the service of society, is committed to the production and dissemination of knowledge and to the creation and transmission professional, scientific, technological, and artistic culture and knowledge, through the articulation of study, teaching, research and experimental development.

Located in the city of Guarda, the IPG has 3 campuses, 4 schools, 3 libraries and 11 research laboratories, as well as 41 Higher Professional Technical Courses, 25 Bachelor´s Degrees, 10 Master´s Degrees, 2 Postgraduate Degrees and 3 Postgraduate Degrees.

It has a staff of 135 teachers with a doctorate, 78 teachers with a master´s degree and 39 senior technicians [1].

1.3 Job Objectives

During the internship, from 05 February to 24 June 2024, a broad spectrum of activities was addressed, including research and implementation in the area of machine learning.

In the line of work focused on research in machine learning techniques, where approaches such as supervised and unsupervised learning, reinforcement learning, and Privacy Enhancing Technologies (PETs) were explored in depth.

In addition, cutting-edge areas such as Federated Learning, which allows collaborative training of models without compromising data privacy, were explored. This technology has been particularly relevant in distributed environments where data confidentiality is a priority.

Another highlight of the internship was the implementation in Python of the Federated Learning technique and different privacy protection techniques, such as differential privacy and Zero-Knowledge Proof. These tools offer innovative solutions to not only ensure data confidentiality, but also to verify the authenticity and validity of transactions.

In parallel, container technologies such as Dockers and Kubernetes were used to optimize the deployment and management of applications in production environments to realize these practical implementations of federated learning. These tools facilitated the creation of secure and isolated environments, minimizing the risk of attacks and security breaches.

Finally, the MQTT communication protocol was explored in the context of the Internet of Things (IoT) device monitoring and management.

1.4 Structure of the Document

The report begins with a summary in English and Portuguese, as well as a list of acronyms and abbreviations.

The first chapter corresponds to the introduction, which describes the company where the internship took place and the institution where I completed my training as a Professional Technician in Cybersecurity. It also presents the internship program and describes the overall structure of the report.

The second chapter, corresponding to the State of the Art, describes and defines the research areas that were required in the course of the internship.

In the third chapter, the development tools are described, indicating the important tools that were necessary for the implementation of the objectives.

In the fourth chapter, which corresponds to the work done during the internship, all the projects and the steps involved are described.

Finally, the conclusion, a critical reflection on the practices carried out, takes stock of the limitations and problems encountered during the internship period.

2 State of the Art

The significant technological advancements and innovations made in the field of Machine Learning (ML) in recent years have not only revolutionized the current industry but have also created new opportunities for research and development in countless fields. This review of the state of the art will define and contextualize ML and present.

A number of machine learning techniques that stand out, including Supervised and Unsupervised Learning, as well as more complex techniques such as Reinforcement Learning and Deep Learning, and cutting-edge techniques such as Federated Learning. Because of the complexity of the last two techniques, a separate section will be dedicated to research on these areas. Given the challenges of Machine Learning, a separate section is dedicated to better understand overfitting and regularization.

The state of the art concludes with tools to improve data privacy and security, including Homomorphic Encryption (HE), Differential Privacy (DP), Zero-Knowledge Proofs (ZKP) and Multi-Party Computation (MPC).

2.1 Introduction to Machine Learning

2.1.1 Definition and Context

Machine Learning is a discipline within the field of artificial intelligence that focuses on the development of algorithms and models that enable computational systems to learn from data and improve through experience without explicit programming for each task. Instead of relying on pre-programmed rules and decisions, machine learning systems can adapt and adjust their behavior by identifying patterns in input data.

The machine learning process typically involves using datasets to train models, where these models are iteratively adjusted and optimized using optimization algorithms. During training, the model seeks to minimize a loss or error function, which involves finding the parameters that best fit the observed data.

Machine Learning is applied in a wide range of fields and sectors, from fraud detection in financial transactions to medical diagnosis, online product recommendations, autonomous driving, natural language processing, and much more. Its growing popularity and applications have been driven by advances in data storage technologies, computational power, and sophisticated algorithms.

By understanding the context and definition of Machine Learning, the different approaches, techniques, and challenges that make up this exciting field of study cab be explored more thoroughly[2].

2.1.2 Brief history and evolution

The development of Machine Learning has been driven by significant advancements in theory, algorithms, and computational technology over the decades. The following summary notes the key milestones in the evolution of the field:

1950s: The early work in machine learning emerged in the context of artificial intelligence. Alan Turing proposed the "Turing Test", a criterion for determining whether a machine can exhibit intelligent behavior, laying the groundwork for research in this field.

1960s-1970s: The first machine learning algorithms were developed, such as the Perceptron by Rosenblatt, which laid the foundation for artificial neural networks. However, progress was hindered by computational limitations and the lack of suitable datasets.

1980s-1990s: Significant advances were made in symbolic machine learning, which is based on the manipulation of symbols and logical rules. At the same time, the development of statistical learning algorithms, such as decision trees and neural networks, marked the beginning of the modern era of machine learning.

2000s onwards: The increase in availability of massive digital data, along with advances in computational power and deep learning algorithms, has led to a resurgence of interest in machine learning. Deep learning techniques, such as recurrent neural networks, have demonstrated exceptional performance in computer vision tasks, natural language processing, and other domains.

Today, machine learning is a mature and constantly evolving discipline, with applications in a wide range of industries and sectors. As the field advances, new developments are expected to further drive innovation and practical application of machine learning.

2.2 Types of Machine Learning

2.2.1 Supervised Learning

In Supervised Learning, a training dataset consists of input-output pairs. The input is the features or attributes representing the characteristics of our data, and the output is what we want to predict or classify, also known as a label or target. By providing many examples of data along with the correct responses, and the algorithm attempts to learn patterns in these data to make predictions or decisions about new data it has not seen before.

For example, when working on an email classification problem to differentiate between spam and non-spam, the features could be keywords in the email, and the output would be the labels "spam" or "non-spam".

This training data is used to feed a supervised learning algorithm, such as a logistic regression classifier, a decision tree, or a neural network, among others. During the training process, the algorithm adjusts its internal parameters to minimize the difference between the predicted outputs and the actual outputs in the training set.

Once the model has been trained, its performance can be evaluated using a test dataset, which are data that the model has not yet seen during training. Our objective is to measure its ability to generalize to new data and make accurate predictions [3].

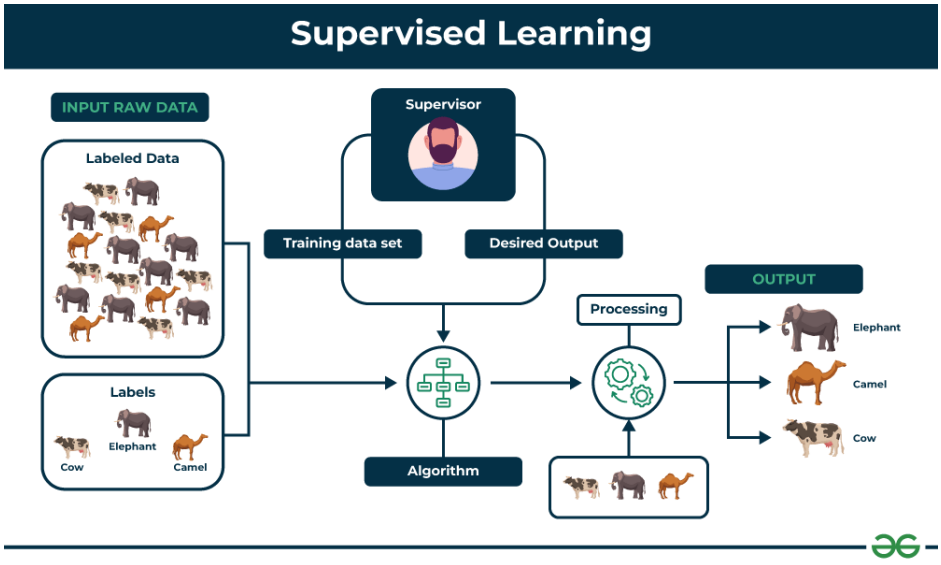


Figure 2.1: Supervised Learning [3]

Among the Supervised Learning algorithms, some of the most common are described.

Linear Discriminant Analysis (LDA): Is a classification algorithm that is used when classes have similar distributions and the features are approximately normally distributed. LDA is applied to find the linear combination of features that best separates the classes. To do so, it seeks a hyperplane that maximizes the distance between the class means and minimizes the dispersion within each class. To classify a new instance, LDA projects the point onto the hyperplane found during training and assigns the class based on which side of the hyperplane it falls on. LDA is computationally efficient and tends to work well when its assumptions are met.

Quadratic Discriminant Analysis (QDA): Is similar to LDA in that they are applied to separate the classes based on features. Nevertheless QDA does not assume equal covariance matrices for all classes. Instead, it allows each class to have its own covariance matrix, making QDA more flexible in terms of the shape of class distributions. Due to this additional flexibility, QDA may be more accurate in situations where classes have very different distributions or when distributions are not approximately normal. However, QDA can become computationally expensive when there are many features, as it needs to estimate a separate covariance matrix for each class.

Linear Regression: Is an algorithm used to predict continuous numerical values. It tries to find the best straight line that fits the data to make predictions.

2.2.2 Unsupervised Learning

In Unsupervised Learning, data analysis is conducted without guidance from labels or known outcomes. Instead of having a labelled dataset, the algorithm receives only inputs and must find patterns or structures in the data by itself. This type of learning is useful when the goal is to explore and understand the underlying structure of the data. It is used in a variety of applications such as customer segmentation, text analysis, social network analysis, image understanding, and more. However, since there are no known outcomes to guide the learning process, interpretation of Unsupervised Learning results can be more subjective and requires careful analysis by domain experts. In general, Unsupervised Learning involves working with unlabeled datasets, meaning there is no explicit information about desired outputs for the inputs [4].

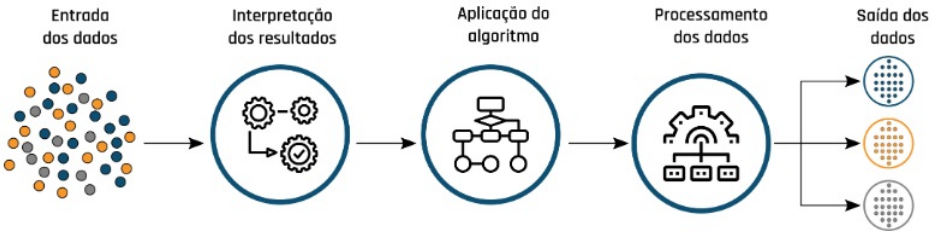


Figure 2.2: Unsupervised Learning [4]

Descriptions of some of the most common types of Unsupervised Learning algorithms, are as follows:

Clustering algorithms group a dataset into clusters or groups of similar elements. The goal is to divide the data into groups where elements within the same group are more similar to each other than to elements in other groups. Examples of clustering algorithms include k-means, Hierarchical Clustering, and DBSCAN.

Dimensionality Reduction algorithms aim to reduce the number of variables in a dataset while retaining most of its information. This can be useful for data visualization or simplifying machine learning problems. Examples of dimensionality reduction algorithms are Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbour Embedding (t-SNE).

Anomaly Detection algorithms are used to identify unusual or atypical patterns in the data, which can be useful for fraud detection, intrusion detection in computer systems, or identifying anomalous data points in a dataset. Some common algorithms include Anomaly Detection Algorithms and distance-based Outlier Detection.

Association algorithms aim to find relationships between variables in a dataset. They are widely used in market analysis and data mining to discover associations between items in large transactional datasets. The most well-known algorithm is the Apriori algorithm.

2.2.3 Reinforcement Learning

Reinforcement Learning is a type of machine learning where an agent interacts with an environment, learning to take actions to maximize a reward signal over time. Through trial and error it learns which actions to take to maximize a reward signal over time. In this machine learning paradigm based on the concept of exploration and exploitation, an agent interacts with a dynamic environment. During training, the agent must explore different actions to discover which ones lead to higher rewards while also exploiting the knowledge gained to maximize short-term rewards. Essentially, it is like teaching a child to play a game[5] [6].

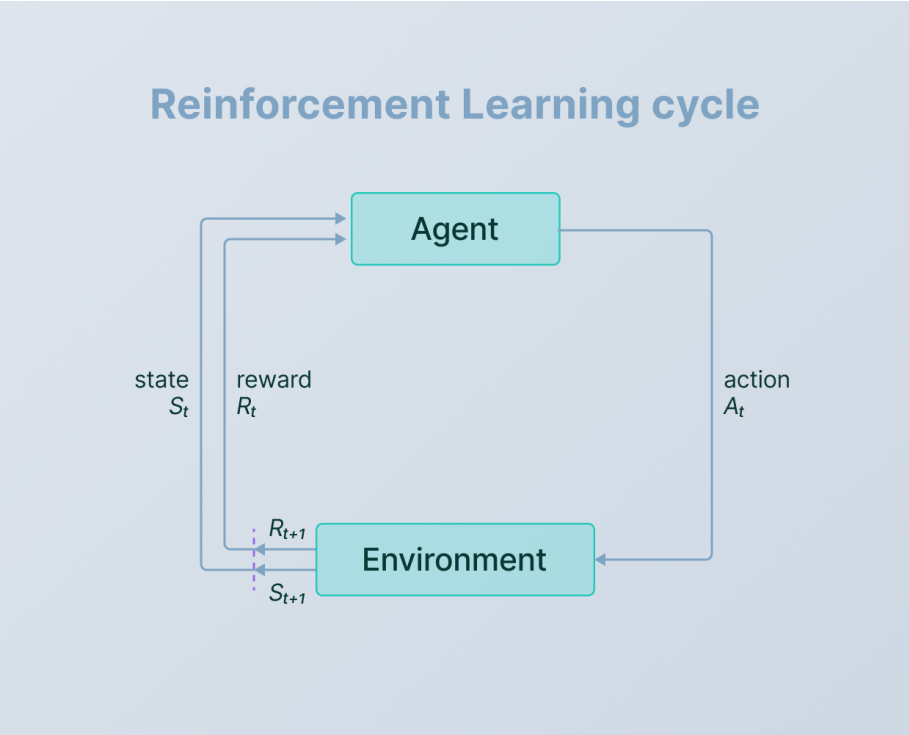


Figure 2.3: Reinforcement Learning [7]

From the research, some of the most common types of reinforcement learning can be described as follows:

Value-based Learning is a useful approach, when the agent's objective is to learn the optimal value function, which assigns a value to each state or state-action and represents the expected utility of being in that state or taking that action. Examples of value-based learning algorithms include Q-learning, SARSA, and Deep Q-Network (DQN).

Policy-based Learning is an approach where the agent directly learns a policy that specifies which action to take in each state, without explicitly estimating the value function. Examples of policy-based learning algorithms include REINFORCE, Trust Region Policy Optimization (TRPO), and Proximal Policy Optimization (PPO).

Actor-Critic Learning combines elements of value-based and policy-based learning. The actor represents the policy and learns to make decisions, while the critic evaluates the policy and provides feedback on how good it is. Examples of actor-critic learning algorithms include Advantage Actor-Critic (A2C) and Asynchronous Advantage Actor-Critic (A3C).

Imitation Learning describes the way the agent learns from examples of humans or from experts, rather than exploring the environment by itself, especially. When the task is complex or dangerous to explore randomly. Examples of imitation learning algorithms include Dataset Aggregation (Dagger) and Generative Adversarial Imitation Learning (GAIL).

Model-Based Reinforcement Learning describes the approach when the agent learns a model of the environment and uses this model to plan future actions and make decisions. This can improve exploration efficiency and enable more informed decision-making. Examples of model-based reinforcement learning algorithms include Dyna-Q and Model Predictive Control.

2.3 Deep Learning

Deep Learning is a branch of Machine Learning that utilizes artificial neural networks to learn complex patterns and representations in data. These deep networks are capable of automatically learning as more data is provided and are especially effective in tasks such as speech recognition, natural language processing, and computer vision. Deep Learning involves at least two types of neural networks - convolutional and recurrent - in pre-trained neural network architectures as well as a class of artificial neural networks called autoencoders. [8].

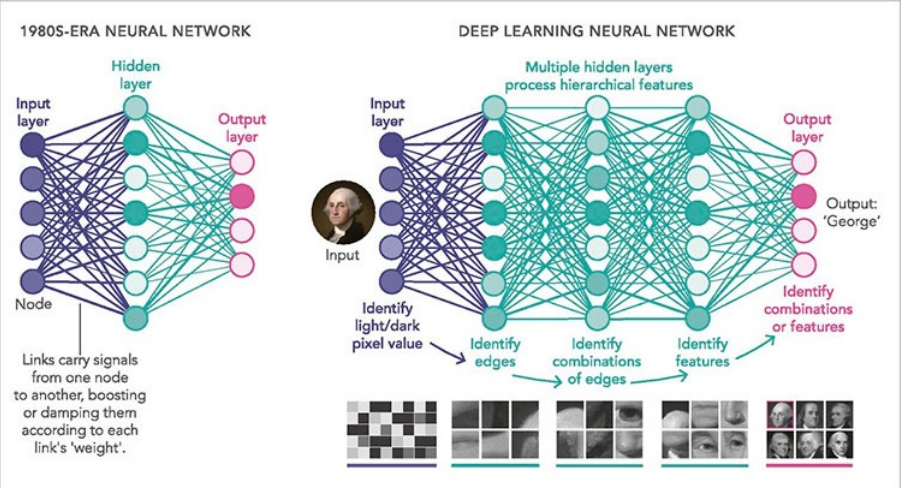


Figure 2.4: Deep Learning [9]

2.3.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a specialized type of artificial neural network designed specifically to process structured data in the form of grids, such as images. They are composed of multiple layers of artificial neurons arranged in a layered architecture. The distinctive feature of CNNs is their ability to directly learn relevant features from input data using convolutional filters.

When repeatedly applied over the input image, they can detect local patterns, such as edges, textures, and shapes. As the network is trained, the weights of these filters are automatically adjusted to maximize the network's ability to recognize relevant features in the input data. In addition to convolutional layers, CNNs typically include pooling layers to reduce the dimensionality of the data and fully connected layers for final classification.

CNNs have demonstrated exceptional performance in a variety of computer vision tasks, such as image classification, object detection, and semantic segmentation, and have been widely adopted in real-world applications due to their effectiveness and efficiency.

2.3.2 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a type of artificial neural network designed to model sequences of data, where the output at a given time depends not only on the current input but also on previous inputs in the sequence. Unlike traditional feedforward neural networks, which have unidirectional connections between layers, RNNs contain recurrent connections that allow them to maintain and update an internal state as they process the input sequence.

This ability to maintain memory over time makes them particularly well-suited for a wide range of natural language processing applications, such as language modeling, machine translation, and text generation. However, traditional RNNs may suffer from gradient problems that hinder effective learning of long-term dependencies in sequences.

To address these limitations, more sophisticated variants of RNNs have emerged, such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), which are capable of learning and remembering long-term dependencies more effectively. These enhanced architectures have demonstrated exceptional performance in a variety of natural language and sequential processing tasks.

2.3.3 Pre-trained Neural Network Architectures

Pre-trained neural network architectures are models that have been trained on large datasets either in a supervised or unsupervised manner to learn useful data representations. These pre-trained models have become widely available due to the increasing popularity of deep learning and the phenomenon of transfer learning.

Transfer learning is a technique that leverages the knowledge learned by a model on one task to improve performance on a related task. By using pre-trained neural network architectures as starting points, researchers and professionals can accelerate the training process and achieve high-quality results with smaller datasets and fewer computational resources.

Pre-trained neural network architectures are available in a variety of domains and applications, including computer vision, natural language processing, and text generation.

2.3.4 Autoencoders

Autoencoders are a class of artificial neural networks used to learn compact and efficient representations of input data in an unsupervised manner. They consist of two main parts: an encoder that maps the original input to a lower-dimensional latent representation and a decoder that reconstructs the original input from this latent representation.

The goal of an autoencoder is to minimize the difference between the original input and the reconstruction generated by the decoder, forcing the model to learn to capture the most relevant and distinctive features of the input data in the latent representation. This ability to learn data representations automatically and efficiently makes them useful in a variety of applications, including data preprocessing, dimensionality reduction, and data generation.

Autoencoders can have different architectures, such as simple autoencoders, convolutional autoencoders, and variational autoencoders, each with their own characteristics and specific applications. In addition to their utility as unsupervised learning tools, autoencoders are also used in semi-supervised and generative learning techniques, where they can generate new data samples similar to the original training data [10] [11].

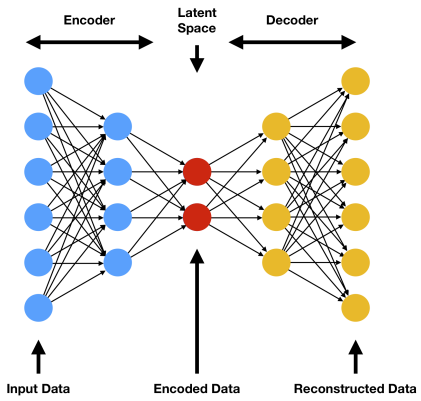


Figure 2.5: Autoencoders [11]

2.4 Challenges in Machine Learning

2.4.1 Overfitting

Overfitting refers to a model that fits the training data too well. This is a concept in data science occurs when a statistical model fits exactly to its training data. When overfitting occurs, the algorithm fails to perform accurately on new, unseen data, undermining its intended purpose,

It occurs when a model learns the details and finds the noise of unwanted random variations or disturbances in data, signals, or processes in the training data to the point where it negatively impacts the model's performance on new data. This means that the noise or random fluctuations in the training data are picked up and learned as concepts by the model. The problem is that these concepts do not apply to new data and negatively impact the model's ability to generalize [12] [13].

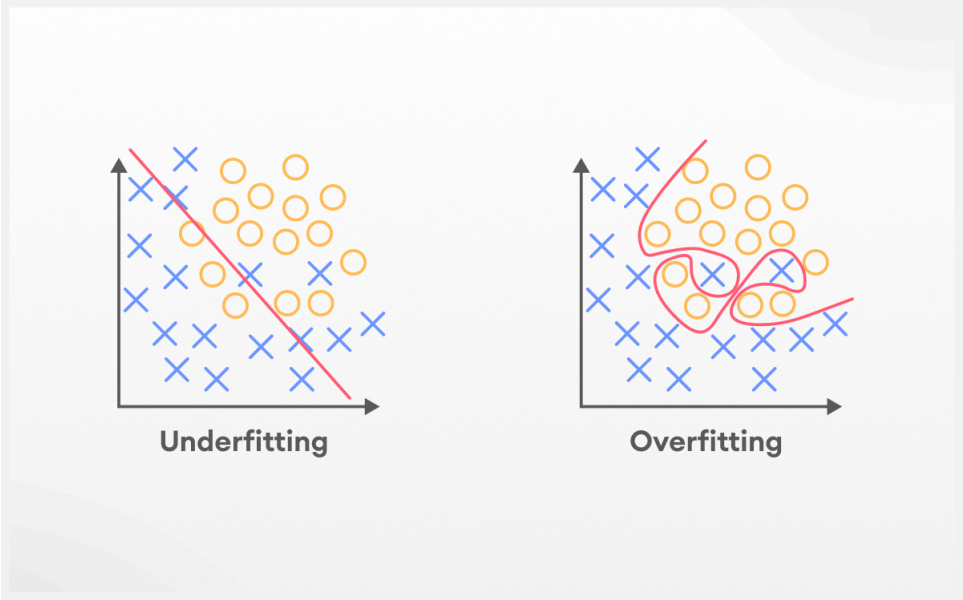


Figure 2.6: Overfitting [13]

2.4.2 Regularization and Overfitting Reduction Techniques

Six of the techniques to reduce regularization and overfitting are indicated.

Early stopping pauses training before the model starts learning the noise within it. This approach runs the risk of stopping the training process too early, which can lead to the opposite problem of underfitting.

Train with more data would be a more effective method when clean and relevant data is injected into the model. Otherwise, you could simply keep adding more complexity to the model, which would also cause overfitting.

Data Augmentation works well in moderation when noisy data is added to make a model more robust.

Feature Selection, commonly confused with dimensionality reduction (see Unsupervised Learning above) is the process of identifying the most important features within the training data and then removing irrelevant or redundant ones.

Regularization techniques make sense to reduce the number of features when model is too complex. But when you do not know which features to remove from the model during the feature selection process, regularization methods can be particularly useful. Regularization applies a "penalty" to input parameters with the largest coefficients, subsequently limiting the amount of variance in the model. While there are various regularization methods, such as Lasso regularization, Ridge regression, and dropout, they all aim to identify and reduce noise within the data.

Ensemble methods, commonly used to reduce variance within a noisy dataset consist of a set of classifiers, such as decision trees, and their predictions are aggregated to identify the most popular outcome. The most well-known ensemble methods are bagging and boosting. In bagging, a random sample of data is selected from a training set with replacement, meaning individual data points can be chosen more than once. After generating several data samples, these models are trained independently and, depending on the type of task, whether regression or classification, most of these predictions provide a more accurate estimate.

2.5 Technologies for Privacy Enhancement

Privacy enhancement is based on a set of technologies and techniques aimed at protecting the privacy and security of data, enabling organizations to collect, store, process, and share information securely and confidentially. These technologies are especially relevant in a world where data is commonly collected and used, and where data privacy and security are growing concerns. The state of the art for privacy enhancement includes homomorphic encryption, differential privacy, zero-knowledge proofs, and multi-party computation.

2.5.1 Homomorphic Encryption

HE is an advanced cryptographic technique that allows mathematical operations to be performed on encrypted data without the need to decrypt them first. This means that calculations can be performed on encrypted data, encrypted results obtained, and then the results decrypted to obtain the correct answer. This is useful for protecting data privacy, especially in scenarios where data is sensitive and confidential.

This type of encryption is highly relevant in situations where data privacy and security are critical, such as in cloud computing applications, sensitive data analysis, and processing of confidential information. By allowing data to remain encrypted throughout the calculation process, homomorphic encryption helps mitigate concerns about data privacy and security [14] [15] [16].

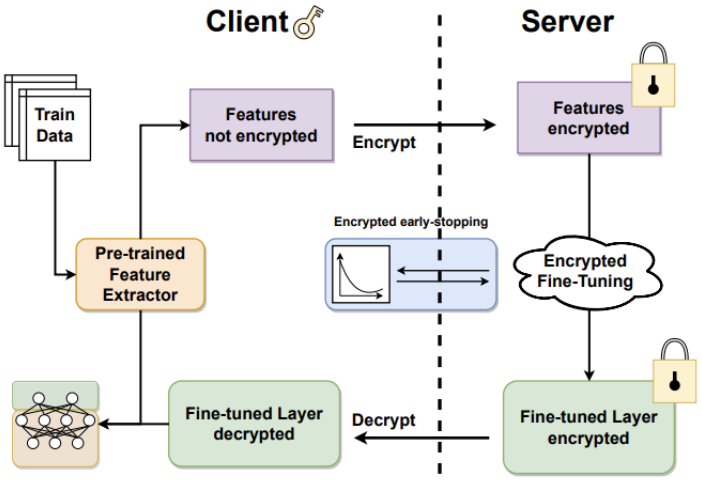


Figure 2.7: Homomorphic Encryption [17]

Each of the types of homomorphic encryption offers, its own characteristics and different levels of security:

Fully Homomorphic Encryption (FHE) allows any mathematical operation to be performed on encrypted data, including addition and multiplication, as well as their combinations. Although extremely powerful, FHE is generally slower and consumes more computational resources than other types of homomorphic encryption.

Partially Homomorphic Encryption (PHE) allows only a specific type of mathematical operation to be performed on encrypted data, such as addition or multiplication, but not both. However, partially homomorphic operations tend to be more efficient in terms of performance than FHE.

Somewhat Homomorphic Encryption (SHE) allows a limited number of mathematical operations to be performed on encrypted data. Although not as flexible as FHE, SHE can offer a trade-off between performance and functionality.

Cryptography and information security are actively engaged in HE research and development, resulting in more practical and efficient techniques and algorithms. However, challenges such as computational efficiency and security against attacks still need to be addressed to make homomorphic encryption more widely adopted in real-world applications.

2.5.2 Differential Privacy

DP is a fundamental concept in data security and privacy, focusing on ensuring that data analysis does not reveal private or identifiable information about specific individuals. The main idea behind differential privacy is to introduce controlled "noise" into the data before performing statistical analyses, so that the results remain accurate and useful without compromising privacy.

Essentially, differential privacy allows organizations to share aggregated statistical information about sensitive datasets without disclosing information about specific individuals who may be included in those datasets [18] [19].

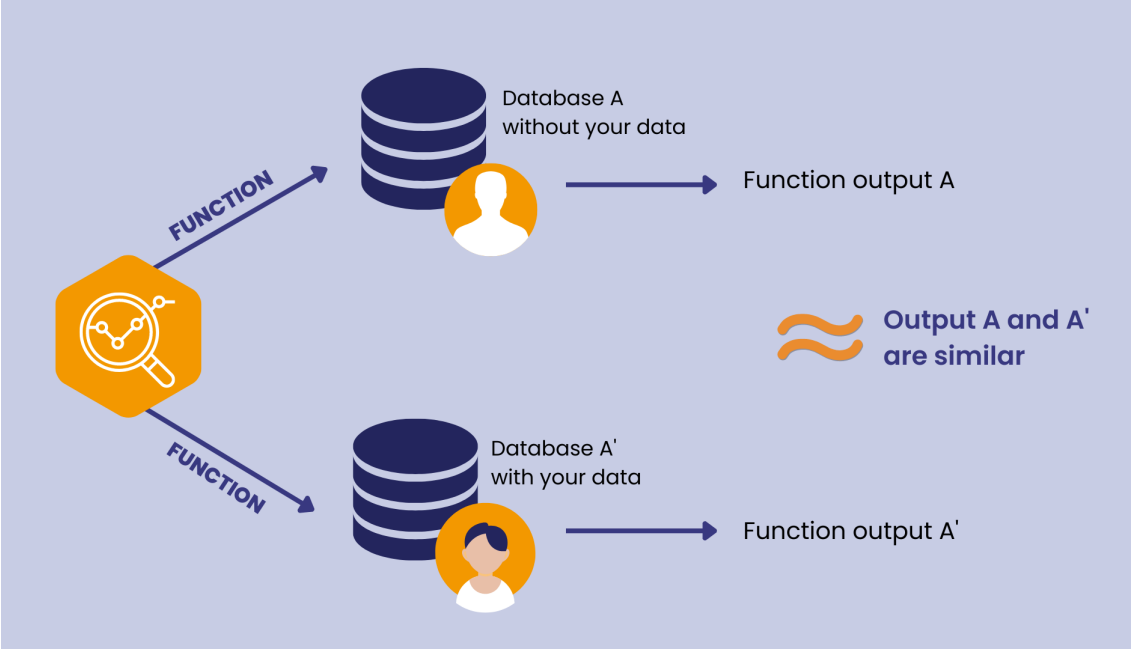


Figure 2.8: Differential Privacy [18]

Some of the key points about differential privacy involve the following aspects:

Controlled Noise: By introducing noise into the data before statistical analysis, differential privacy protects individuals' identities and sensitive details in their data. This noise is carefully calibrated to ensure that the aggregated results remain useful and accurate even in the presence of noise.

Formal Threat Model: Differential privacy provides a formal model for quantifying how much a statistical analysis can reveal about any specific individual within the dataset. This helps establish clear limits on the amount of information that can be extracted from the data without compromising privacy.

Applicability in Various Contexts: Differential privacy is applicable in a variety of scenarios, including health data analysis, social research, mobile data collection, and more. Any situation where individuals' privacy needs to be protected during data analysis can benefit from the principles of differential privacy.

Balance between Privacy and Utility: One of the challenges of differential privacy is finding the right balance between protecting individuals' privacy and ensuring that the analysis results are still useful enough to be meaningful for researchers, analysts, and other data users.

2.5.3 Zero-Knowledge Proofs

ZKP are a class of cryptographic protocols that allow one party to prove to another party that they possess certain knowledge or secret without revealing any information about the content of that knowledge. In other words, a zero-knowledge proof enables a party to demonstrate that they have certain information without revealing the information itself [20] [21].

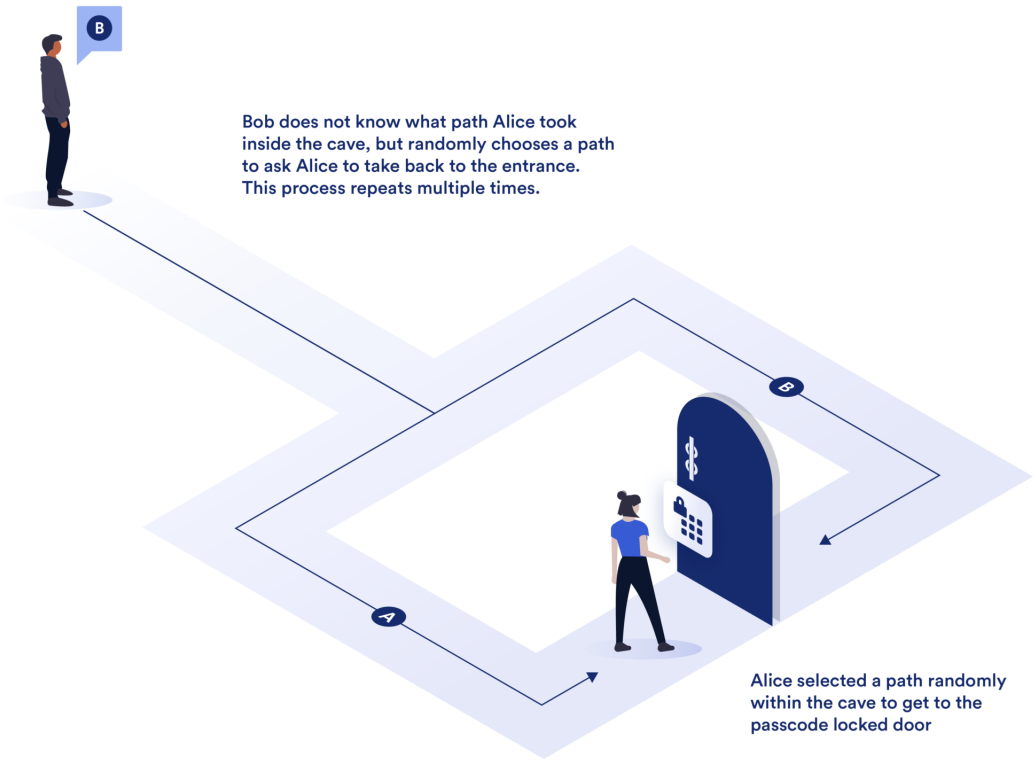


Figure 2.9: Zero-Knowledge Proof [20]

These protocols are fundamental in situations where it is necessary to verify the authenticity or possession of certain knowledge without exposing sensitive details. Here are some important concepts about ZKP:

Knowledge Confirmation: A ZKP not only confirms that the prover possesses the knowledge but also demonstrates that they truly know the information in question and are not simply guessing or deceiving in any way.

Complete Secrecy: One of the most important aspects of ZKP is that they ensure no additional knowledge is transferred beyond the validity of the knowledge itself. Even if a party proves that they possess certain information, they do not reveal anything more about that information, making the protocol extremely secure and preserving privacy.

Practical Applications: ZKP have a wide range of applications in information security, cryptocurrencies, authentication, and more. For example, they are used in electronic voting systems to ensure that votes are counted correctly without revealing the identity of the voters. Additionally, they are employed in blockchain to conduct transactions securely and privately.

Types of Proofs: ZKP can be Interactive, where the interactions are amongst the involved parties, and Non-Interactive, where on party proves their knowledge in a non-interactive manner, which is useful in many practical applications.

2.5.4 Multi-Party Computation

MPC is a field of cryptography that enables multiple parties to compute a function over their input data without revealing confidential information to other involved parties. MPC allows multiple entities to perform collaborative computations on their combined data while preserving the privacy of each individual dataset [22] [23].

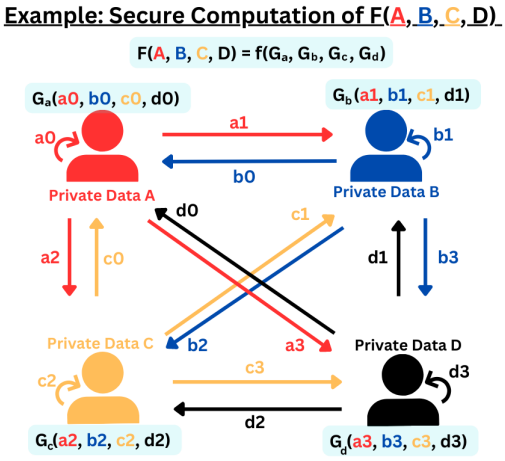


Figure 2.10: Multi-Party Computation [24]

Some important characteristics of Multi-Party Computation are as follows:

Preserved Privacy: The central goal of MPC is to ensure that input data remains private throughout the computation process. Each party contributes its input without revealing information about it to other parties.

Security against Malicious Entities: MPC is designed to function even when some of the parties involved in the computation are dishonest or potentially malicious. This is achieved through cryptographic protocols that ensure the accuracy of computation results regardless of the behavior of the involved parties.

Diverse Applications: MPC has a wide range of applications in areas such as finance, healthcare, and research. For example, it can be used for joint data analysis among different organizations without revealing sensitive details about each organization's individual data.

Computational Efficiency: A challenge in MPC is to ensure that computations can be performed efficiently, especially when involving large datasets or a large number of parties. Ongoing research aims to develop more computationally efficient MPC protocols.

Types of Computation: There are different types of MPC, including two-party secure computation and multi-party secure computation. Each type of MPC is designed to meet different security and privacy requirements depending on the application context.

2.6 Federated Learning

2.6.1 Definition and Basic Principles

Federated Learning, a decentralized approach to machine learning training, is a way of training AI models without anyone having access to the data, providing privacy and security, and a means of unlocking insights to power new AI applications. The exchange of data between client devices and global servers is not required; instead, raw data remains on client devices, and only the model parameters are shared between the devices. Federated Learning is a distributed machine learning approach that enables training machine learning models on edge devices locally. Training on these client devices, such as cell phones, where only model updates, not raw data, are shared with a central server. This means there is no need to transfer data to a central server [25] [26] [27].

Some of the specific characteristics of Federated Learning regarding each of these areas of concern are as follows:

Privacy: Federated Learning enables training to occur locally on the device, avoiding potential data breaches.

Data security: Only encrypted model updates are shared with the central server, ensuring that raw data remains secure. Additionally, the Secure Aggregation Principle allows decryption of the aggregated results.

Access to heterogeneous data: Data is stored on different devices, locations, and organizations and is not available to be transferred to a central server. This way, models can be trained with sensitive data, such as health data, without the need to transfer data to a central server.

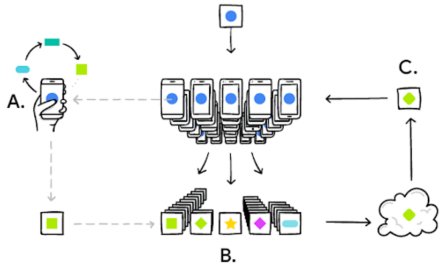


Figure 2.11: Federated Learning [25]

2.6.2 Applications and Use Cases

The generic federated learning process illustrated in figure 2.11 involves a central server storing a base model, which is then shared with client devices. Each client device trains the model with its local data and sends the model updates back to the central server using secure aggregation techniques. The central server aggregates the model updates from all client devices and updates the base model. This process is repeated until the model achieves the desired accuracy.

This model combines and averages different inputs to generate new insights. Since the data is collected from diverse sources, there is greater diversity in the data, which can lead to a more robust and accurate model.

2.6.3 Challenges and Security Considerations

Efficiency in Communication: Message transfer becomes slow due to various reasons such as network delays, packet loss, network congestion, bandwidth limitations, resource scarcity, or geographical location. To maintain efficient channels, the total number of message passes and the size of a message in a single pass should be reduced. This can be achieved through:

Local Update Methods: By reducing the number of local updates, mini-batch size, and the number of local iterations.

Model Compression Schemes: By reducing the model size, such as model compression, model quantization, and model pruning.

Decentralized Training: By reducing the frequency of communication between devices. However, a large number of devices are required to achieve significant results.

Privacy and Data Protection: Privacy and data security are some of the biggest concerns with federated learning. While local data remains on the user's device, there is a risk of information being revealed from shared model updates. To mitigate these risks, some common privacy-preserving techniques that can address this issue include:

System Heterogeneity: The heterogeneity of client devices can lead to significant variation in model quality. Heterogeneity may be due to differences in computing capability, network connectivity, power, and storage resources. To mitigate these challenges, it is necessary to:

Device Selection: Client device selection should be based on computing capability, network connectivity, and power.

Adaptive Learning: The model should be trained according to the computing capability and connectivity of devices.

Heterogeneous Federated Learning: Model training should be distributed according to the computing capability and connectivity of devices.

2.6.4 Advantages and Disadvantages of Federated Learning

The advantages of Federated Learning are important:

Enhanced Privacy: Raw data remains on client devices, reducing the risk of data breaches. It improves data privacy as raw data is not transferred to a central server. Network Efficiency: Reduces network load as only the model parameters are shared between client devices and the central server.

Learning on Decentralized Devices: Enables model training on client devices such as cell phones without the need to transfer data to a central server. Continuous Adaptation: The central model can be continuously trained with updated data, adapting to changes in user patterns and preferences over time. Enhanced Security: Reduces the risk of data breaches as raw data remains on client devices, and only model parameters are shared between client devices and the central server.

Despite these advantages, it is wise to be aware of the following disadvantages

Model Management Complexity: Managing multiple distributed models can be complex. Coordination and synchronization of models may pose challenges, especially in heterogeneous environments.

Frequent Communication: Frequent communication between client devices and the central server can lead to network delays and congestion.

Convergence Issues: Model convergence may be slower compared to centralized approaches due to frequent communication and aggregation of distributed updates.

Sensitivity to Data Heterogeneity: If data on local devices differs, aggregation of model updates may lead to a final model of low quality.

Local Overfitting: Local models may overfit to local data, especially if the amount of data on a device is small.

3 Development Tools

3.1 Visual Studio Code

Visual Studio Code is a source code editor developed by Microsoft for Windows, Linux, macOS and Web. It includes support for debugging, integrated Git control, syntax highlighting, smart code completion, fragmenting and code refactoring. It is also customisable, so users can change the editor theme, keyboard shortcuts and preferences. It is free and open source, although the official download is under proprietary software and includes features customized by Microsoft.

Visual Studio Code is based on Electron, a framework used to implement Chromium and Node.js as desktop applications running on the Blink design engine. Although it uses the Electron framework, the software does not use Atom but instead employs the same editor component (Monaco) used in Visual Studio Team Services (formerly Visual Studio Online) [28] [29].

3.2 Python

Python is a powerful and easy-to-learn programming language. It has efficient high-level data structures and a simple but effective object-oriented programming system. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and application development in many areas, for most platforms.

The Python interpreter and extensive standard library are freely available in source code and binary form for most platforms from the Python website at, /www.python.org/ The same site also contains distributions and references to many free third-party Python modules, programs, tools and additional documentation.

The Python interpreter is easily extended with functions and data types implemented in C or C++ (or other languages that allow them to be called from C). Python is also suitable as a language for extending modifiable applications [30] [31].

3.3 TensorFlow

TensorFlow is one of the most popular and widely used open-source machine learning libraries developed by the Google Brain Team. It provides a flexible and comprehensive framework for building and training machine learning and deep learning models [26] [32].

Some key aspects of TensorFlow include the following:

Data Modelling: TensorFlow represents data and mathematical operations as computational graphs. Nodes in the graph represent mathematical operations, while edges represent the data (tensors) flowing between these operations.

Flexibility: TensorFlow offers flexibility to work at different levels of abstraction, from building models from scratch to using high-level Application Programming Interface (APIs) like Keras, which simplify the process of model building and training.

Support for Different Devices: TensorFlow is designed to support execution on various platforms, including CPUs, GPUs, and Tensor Processing Unit (TPUs), allowing users to leverage the computational power of diverse hardware.

Rich Ecosystem: TensorFlow has a rich ecosystem of tools and extensions that simplify tasks such as data visualization, preprocessing, model evaluation, and deployment in production. Additionally, there are libraries and frameworks built on top of TensorFlow, such as TensorFlow Extended (TFX) for machine learning pipelines, TensorFlow.js for machine learning in web browsers, and TensorFlow Lite for deployment on mobile and embedded devices.

Community Support: TensorFlow has an active and constantly growing community of developers, researchers, and enthusiasts who contribute code, tutorials, and educational resources, making it easier for new users to learn and engage with the library.

Practical Applications: TensorFlow is widely used in a variety of machine learning and deep learning applications, including speech recognition, natural language processing, computer vision, automatic translation, and data analysis, among others.

3.4 Flower

Flower (Federated Learning with TensorFlow) is an open-source library that originated from a research project at the University of Oxford. Now managed by adap gmbh, a German distributed AI company, Flower aims to facilitate the implementation of federated learning in TensorFlow-based projects [33].

Some important points about Flower cover most of the frequently asked questions:

Simplified Federated Learning: Flower provides a simplified and consistent interface for training machine learning models federatedly. It abstracts the complexity of managing communications, coordinating distributed training, and aggregating models so that the, developers can focus more specifically on the model logic.

TensorFlow Support: Flower is specifically designed for integration with TensorFlow, one of the most popular machine learning libraries. It enables developers to federatedly train TensorFlow models easily, leveraging the familiar tools and resources of the TensorFlow library.

Flexibility and Customization: Flower offers a variety of options for customizing and adjusting the behavior of federated learning according to the specific requirements of the project. Developers can implement custom aggregation algorithms, define communication strategies, and configure federated training policies to meet the individual needs of the application.

Efficiency and Scalability: By distributing model training among edge devices, federated learning can improve computational efficiency and reduce the amount of data that needs to be transmitted over the network. This is especially useful in scenarios where data is sensitive or restricted, and where user privacy is a concern.

Active Community: Flower is an open-source library with an active community of developers and contributors. As a result, This means that users can benefit from regular updates, bug fixes, and ongoing support from the community.

3.5 Docker

Docker is an open-source platform that facilitates the creation, deployment, and execution of applications in containers. Containers are a form of virtualization that allows packaging an application along with all its dependencies and necessary configurations into an isolated unit that can be consistently run in any environment that supports Docker [34].

Some important points about Docker are as follows:

Containers: Docker containers are lightweight and portable units that include an application's code, its libraries and dependencies, and all necessary configurations to run it. They isolate the application from its execution environment, ensuring consistency and predictability across different environments, from development to production.

Docker Images: A Docker image is a kind of "template" for creating containers. It contains all the information needed to run an application, including the base operating system, the application's libraries and dependencies, and the startup commands. Docker images are built from definition files called Dockerfiles.

Container Orchestration: Docker also provides tools for managing and orchestrating containers at scale, such as Docker Swarm and Kubernetes. These tools allow deploying and scaling applications in a cluster of machines efficiently and reliably.

Portability and Consistency: One of Docker's main advantages is its portability. Since containers encapsulate all the application's dependencies, they can be run consistently in any environment that supports Docker, from developer laptops to production servers and public clouds.

Resource Efficiency: Docker containers share resources from the underlying operating system, making them lighter and more resource-efficient than traditional virtual machines. This allows running more applications on a single physical machine, reducing costs and optimizing resource usage.

Industry Standard: Docker has become an industry standard for packaging and distributing applications, widely adopted by companies and organizations of all sizes in various fields, from startups to large corporations.

3.6 Kubernetes

Kubernetes is an open-source platform for automating the deployment, scaling, and management of applications in containers. Developed by Google and later released as an open-source project, Kubernetes offers a robust and flexible environment for orchestrating containers in production environments [35].

Some important characteristics of Kubernetes are as follows:

Container Orchestration: Kubernetes simplifies the process of deploying and managing containers at scale. As a result containers can be defined to reflect how they should be deployed, scaled, and maintained, ensuring that applications are always available and running correctly.

Microservices-Based Architecture: Kubernetes is designed with a microservices-based architecture, meaning that each component is designed to be modular and independent. This facilitates scalability, maintenance, and extensibility of the platform.

Automation and Declarative Description: With Kubernetes, the desired state of the system can be described in a YAML configuration file, known as a manifest. Kubernetes then automates the process of provisioning and managing resources to ensure that the actual state of the system matches the declared state.

Horizontal and Vertical Scalability: Kubernetes supports horizontal and vertical scalability so that, applications can be dynamically scaled based on workload. This ensures that applications can handle traffic spikes without service interruptions.

Monitoring and Logging Resources: Kubernetes provides built-in resources for monitoring the health of applications and clusters, as well as for collecting and analyzing container logs. This helps administrators identify and resolve issues proactively.

Compatibility with Multiple Cloud Providers: Kubernetes is compatible with a wide variety of cloud providers as well as on-premise environments. This allows development and operations teams to adopt an infrastructure-as-code approach and migrate applications between different environments seamlessly.

Active Community and Rich Ecosystem: Kubernetes has an active community of developers and users, resulting in a rich ecosystem of tools, plugins, and extensions. This makes it easier for organizations to extend and customize Kubernetes to meet their specific needs.

3.7 Message Queuing Telemetry Transport

MQTT, which stands for Message Queuing Telemetry Transport, is a lightweight and low-bandwidth communication protocol designed for devices with limited resources and unstable network connections. Developed jointly by IBM and Arcom (now Eurotech) in 1999, MQTT has since evolved as an open standard under the care of the OASIS Organization [36].

The key Features of MQTT include the following:

Lightweight Protocol: MQTT was designed to be lightweight and resource-efficient, making it suitable for devices with constraints on power, memory, and bandwidth, such as IoT sensors and embedded devices.

Publish/Subscribe Model: MQTT operates on a publish/subscribe (pub/sub) model. In this model, devices subscribe to "topics" of interest and receive messages published on those topics. This allows asynchronous communication between devices.

Network Topology Flexibility: MQTT is suitable for a variety of network topologies, including point-to-point, star networks, and mesh networks. This makes it versatile for implementation in various IoT scenarios and messaging applications.

Quality of Service (QoS): MQTT supports three levels of QoS to ensure reliable message delivery between devices. These levels include QoS 0 (at most once delivery), QoS 1 (at least once delivery with acknowledgement), and QoS 2 (exactly once delivery).

Low Data Overhead: MQTT minimizes data overhead compared to other communication protocols, as it uses a fixed-size header and does not require a large data exchange to establish or maintain a connection.

Security: MQTT can be implemented with additional security measures, such as Secure Sockets Layer (SSL)/Transport Layer Security (TLS)for data encryption and certificate-based authentication. This ensures that data transmitted by MQTT is protected against unauthorized access.

MQTT Brokers: In the MQTT architecture, devices communicate through a centralized or distributed "broker." The broker receives messages published by devices and forwards them to the corresponding subscribed devices.

Wide Adoption: MQTT has gained widespread adoption in the IoT industry and is used by many companies and organizations in a variety of applications, from remote monitoring to home automation and energy management.

3.7.1 Mosquitto

Mosquitto is a popular open-source server for the MQTT protocol. MQTT is a lightweight messaging protocol designed for communication between IoT devices, embedded systems, and mobile applications in networks with limited bandwidth and high latency [37].

Mosquitto was developed by Roger Light in 2009. Since then, it has been maintained by the open-source community and has a wide user base and contributors. It uses the MQTT protocol described in the previous section. It is extremely bandwidth and resource-efficient, making it ideal for IoT applications and sensor networks.

Notably Mosquitto features the following advantages:

- Support for MQTT protocol v3.1 and v3.1.1

- Implementation in various programming languages, making it lightweight and efficient.

- Support for SSL/TLS for secure connections.

- Username and password-based authentication.

- Access Control List (ACL) based access control.

- Message forwarding between different Mosquitto servers or between MQTT and other messaging protocols.

- Scalability and performance to handle large volumes of messages in IoT networks. Save to grepper

Installation and Usage: Mosquitto is available on most Linux distributions and can also be compiled from source on other platforms. It can be used from the command line or integrated into applications through its API.

Use Cases: Mosquitto is used in a wide range of IoT and embedded systems applications, including home automation, remote monitoring, telemetry, and industrial automation. It is particularly popular in applications where efficient and scalable communication between connected devices is required.

Community and Support: Mosquitto has an active community of users and developers who provide support through forums, mailing lists, and other online communication channels. There is also comprehensive documentation available to assist with the installation, configuration, and use of Mosquitto.

4 Development Work

4.1 Initial Part

The semi-remote internship included in-person work every Thursdays, at the office in Coimbra. During these weekly visits, I presented my progress on the project, resolved doubts and participated in the weekly meeting with the team leader and other staff.

The internship was divided into two phases. The first part, which ran from 05 February 2024 until the first weeks of March, focused on research, learning, and understanding the topics developed and shared in this report in section 2, State of the Art.

For the weekly Thursday meetings, I prepared detailed presentations of the research topics in PowerPoint. Following each presentation, I would answer questions from the team leader as they arose during the presentation. At the end of the meetings, the topics to be investigated for the following week were established.

4.2 Implementation

After the research phase, the first step was to install a virtual machine. I had some guidance in my choices but, ultimately, I chose to use VMware Workstation Player (desktop virtualization application that is available free of charge for personal use) and set up a Debian-based operating system without a graphical environment. Next, I established an SSH connection (Secure Shell and is a network protocol intended primarily for connecting to machines that we access via command line. In other words, with SSH we can connect to servers using the Internet as a means of communication.) between Visual Studio Code (the platform I used for all the Python code implementation) and the Debian operating system.

With the secure connection established, I proceeded to install the libraries needed for the project, damely TensorFlow and Flower, which were previously mentioned in the section on Development Tools.

Once these preparations were completed, the next step involved training a machine learning model using the MNIST dataset [38]. This dataset, which is widely used to train image processing systems, gave me some initial experience in training a machine learning training.

After the first training, the next challenge was to implement the code for Docker. This involved creating a Dockerfile 5.1, which is a text file with the instructions needed to build a container image. In the technological context, a container does not store physical objects, but a stack of applications that, linked together, create the image that is subsequently used. requirements.txt file 5.2 must be created. This text file is used to specify which libraries and their versions are necessary for the project to work correctly. It is an essential file to ensure that all the dependencies and their corresponding versions have been installed in the execution environment.

With the experience gained in the use of Docker containers, my next effort was a broader project based on training for machine learning.

This project was based on capturing real-time network traffic on TCP connection, which would then analzsed by the autoencoder. The process began with first creating a client, a server, a collector and a generator, all coded in Python. Each of these creations are detailed in the four separate sections that follow, but to understand the flow of the activity, let us proceed.

Once this was functional, the next step was the implementation part of the client privacy techniques, which involved Differential Privacy and Zero-Knowledge Proofs with the noknow library [39] [40].

The next step was to implement this code for Kubernetes by creating some .yaml files 5.3. This was tricky because when you create an object in Kubernetes, you must specify the object spec that describes its desired state, as well as basic information about the object (such as its name). When you use the Kubernetes API to create the object (either directly or using kubectl), that API request must include all the information in JSON format in the body of the request. Often, you provide the information to kubectl as a .yaml file. kubectl converts that information to JSON when it makes the API call. In the .yaml file of the Kubernetes object you want to create, you will need to provide the values of at least the following fields:

- **apiVersion**: Which version of the Kubernetes API you are using to create this object

- **kind**: What kind of object you want to create.

- **metadata**: Data that allows you to uniquely identify the object, including a string for the object's.

- name, UID, and optionally the namespace.

You must also specify the spec field of the object but the format of the spec field is different depending on the type of Kubernetes object, and contains nested fields specific to each object. The Kubernetes API Reference can help you find the spec format for each of the objects you can create using Kubernetes.

With the work process clear, the subsections below will describe in detail the activity required to create the client, the server, the generator, and the collector.

4.2.1 Client

To create the client, which is divided into several pieces of code, I carried out the following steps:

The main code is **main.py**, which you can find in Appendix 5.4. This code implements a Federated Learning environment using Tensorflow, Flower and techniques such as Sockets and MQTT for communication and synchronization of processes. It also implements the two privacy techniques ZKP and DP. This way, the client connects to the server, sends and receives model parameters and performs model evaluations.

First the libraries and modules necessary for the correct functioning of the code are imported.

Then we have the Client Types are designated, where the possible client types are defined as, '**NONE**', '**ZKP**', '**DP**' and '**BOTH**'.

Next is the function **get.ip.server**, a function designed to get the IP of the server using the Kubernetes API.

After that, the **send.parameters** function sends model parameters and performance metrics to the server using MQTT.

The function **connect.server** then connects to the server according to the client type and sends performance parameters.

Then there are the functions for creating and managing the processes to connect to the server and evaluate the model. These functions are **create.process.server** and **create.process.evaluate**.

The next function is **connectZKP**, a function that manages the ZKP implementation, generating and sending the cryptographic tests to the server through sockets.

Finally, the **main** function loads the data, creates the model, gets the client type and schedules the connection to the server every 30 seconds. It executes the pending programming indefinitely.

ClassClient.py 5.5 defines several clients for the federated learning environment. Each client is designed for a specific strategy, either network flows or network flows with a specific privacy strategy.

At the beginning of the code you can see the imports of the necessary libraries and modules.

Next are the different types of clients, with their **init**functions, which is the constructor of the class that initializes the model. The **get.parameters** function returns the current weights of the model. The **fit** function, which is the method that trains the model locally using the data provided. The **evaluate** function evaluates the model locally. The fit function is a special case in the **ClientZKPandDP** client where **utils.train.DP.mode** (**self.model, dfNormalized, 10, 0.1**) trains the model by applying differential privacy.

The code **utils.py** 5.6 is related to building, training, and evaluation. The first lines import the libraries and modules needed for the code.

Next is the **training.values** function loads the **MNIST dataset** of handwritten digits, normalizes the images by dividing by 255, and returns the training sets. Then the function **local.evaluate** evaluates the model using test data. It predicts the values and calculates the mean absolute error (MAE). If the MAE exceeds the threshold of 0.5 it is considered a potential threat and the corresponding rows are removed from the CSV file. The **create.model** function creates an **autoencoder** model, using TensorFlow, composed of several dense layers, compiled with the **Adam optimiser** and the mean square error loss function, and trained with the normalized data.

The function **add.noise.to.weights** adds noise to the model weights to provide differential privacy. It iterates over each layer and its input parameters, adding Gaussian noise with a specified standard deviation. Finally the function **train.DP. model** trains the model with differential privacy by adding noise to the weights in each layer.

Finally the **loadProcData.py** 5.7 includes first the import of the libraries and modules needed for the code. The function **load.and.preprocess.data** first loads the training and test data from a CSV file into two pandas **DataFrames**. Then the numerical and categorical columns in the data are identified. Two preprocessing pipelines are then created. For numerical features, missing values are imputed '**StandardScaler**' with the mean and scaled. For categorical features, missing values are imputed using with the most frequent value and encoded using '**OneHotEncoder**'. Columns that are not needed for data analysis are also removed, and normalized data is returned for training and testing.

4.2.2 Server

The server is divided into two codes, which are detailed below:

The main code, **main.py** 5.8, implements a federated learning system that uses ZKP to authenticate clients. It uses multiple technologies and libraries such as Tensorflow, Flower, Sockets, and MQTT for communication and process synchronization.

First the libraries and third party modules needed to operate the system are imported.

For the initial code setup, a name for the database is defined and a **ClientType**' numerator specifies the client types. The '**select.client.type**' function allows the client type to be selected via a user input.

Next, the functions are defined: **define.strategy**, which defines a strategy for the Flower server using '**FedAvg**'; **start.federated.round** function to start federated learning rounds; and **start.flower.server**, which starts the Flower server on a dynamically obtained IP address.

After that, the Wait and Client management functions, which configure and manage client authentication use ZKP and sockets. The **wait.option** and **serverZKP** functions wait for client connections and handle authentication and verification of ZKP tests.

The main function, **main**, initializes the logging configuration, creates the database, selects the client type, starts the server processes and configures the MQTT client. It also initiates federated learning rounds periodically.

4.2.3 Collector

The main objective of this collector code 5.10 is to capture and analyze the data flows of a network interface using the '**NFStreamer**' library, and then save these flow statistics in a CSV file.

The code first has the corresponding library and module imports necessary for proper operation. Then two command line arguments are defined, '**networkInterface**' and '**filter**', a filter to capture TCP traffic.

The next section of the code, '**NFStreamer**' is configured with various parameters to capture and analyze traffic from the specified network interface.

Finally, each captured flow is iterated over, and flow statistics are created and saved to the CSV file.

Inside the collector a custom plugin 5.11 extends the '**NFPlugin**' of the '**NFStream**' library, which is used to add additional features to the network flows captured by the collector.

Some imports and constants that configure the environment and parameters used in the plugin are '**CSV.PATH**', which defines the path to the CSV file where the data will be saved; '**CLUMP.TIMEOUT**', '**BULK.BOUND**', and '**ACTIVE.TIMEOUT**' are parameters to calculate specific network traffic characteristics; and '**CustomPlugin**' processes the captured flows, adding additional metrics.

This architecture allows for detailed customization and analysis of the captured network traffic in real time.

4.2.4 Generator

The code 5.12 generates TCP traffic by simulating visits to different websites. First, there are the corresponding imports. Second is the function '**generateTcpData**', which generates TCP traffic by making random and continuous HTTP requests to a list of websites. Each request is made with an interval between 1 and 3 seconds.

4.3 Results

Once the code development was completed and the network traffic for each of the clients was captured and analyzed, the data obtained was analyzed in detail. In The analysis showed how the implementation of Differential Privacy DP and Zero-knoledge Proof ZKP security systems affected the different training of machine learning models.

4.3.1 Training Perfeormance Without Security

When the training was performed without any type of security system, the results showed that the process was almost instantaneous and did not require intensive use of computer resources. Key metrics analyzed in this scenario included:

Training Time: The total time to complete the training was significantly low.

CPU and Memory Usage: Minimal CPU and memory usage was observed during the training process.

Model Accuracy: The accuracy of the trained model without safety measures was stable and comparable to a standard reference index.

4.3.2 Impact of Differential Privacy

The implementation of the DP introduced a considerable delay in the training process. This is due to the data noise addition techniques required to ensure data privacy. Key points include:

Training time: There was a noticeable increase in training time due to the additional computation required to ensure privacy.

Resource Usage: An increase in CPU and memory usage was observed, indicating a higher computational load.

Privacy vs Accuracy: Although a high level of privacy was maintained, a slight degradation in model accuracy was noted compared to training without security.

4.3.3 Impact of Zero-Knowledge Proofs

By applying ZKP, the training process also showed an increase in the training process and in the use of resources. The ZKP tests provide a high level of security without revealing confidential information. Key results include:

Training Time: Similar to DP testing, ZKP testing significantly increased training time, given that client-server authentication must be done before each round of training.

Resource Usage: CPU and memory usage was high, similar to the scenario with DP.

Security and Efficiency: Training with ZKP ensured data integrity and privacy, but at the cost of increased complexity and resource consumption.

4.3.4 Comparative Analysis

A comparison between the different scenarios (without security, with DP and with ZKP) highlighted the following points:

Efficiency vs. Security: training without security measures was the most efficient in terms of time and resources, but with the lowest level of security. In contrast, both DP and ZKP offered high levels of security, but at the cost of increased complexity and processing time.

Resource Usage: The security techniques significantly increased the use of computational resources, highlighting the need to balance between data security and operational efficiency.

5 Conclusion

The internship at OneSource has been, without a doubt, one of the biggest challenges I have ever faced. The first challenge was the change of sector, moving from mechanics to IT, and specifically to cybersecurity. The second major challenge was the language, as studying in Portuguese, a language that is not my mother tongue, is complicated. This was compounded by English, which I did not initially master well. However, with effort, dedication, and perseverance, I have worked to overcome each obstacle one step at a time.

The internship started positively, allowing me to dedicate time to research so that I could focus and learn a lot about machine learning. I became familiar with the different types of machine learning, such as supervised, unsupervised, and reinforcement learning as well as deep and federated learning, and took the time to understand their inherent challenges. I also gained knowledge about privacy-enhancing technologies, which I had to implement in my code. These included Differential Privacy and Zero-Knowledge Proofs.

The second and longer part of the internship was where the real challenge began. Working with Python gave me the opportunity to improve my programming skills significantly. I also found that working with new tools, such as Docker and Kubernetes, presented a number of problems. The first hurdle was understanding Kubernetes, learning how to create a .yaml file, and configuring the IPs to make everything work correctly.

Although Kubernetes is an area where I can still improve, good learning habits of continuous practice and seeking constant feedback have helped me to gradually overcome most of the challenges. I already knew the importance of resilience and adaptability and through this internship, I saw up close that these characteristics are especially relevant in such a dynamic field as cybersecurity.

In summary, my internship at OneSource has been a deep and multifaceted learning experience. I have developed valuable technical skills and learned to face and overcome significant challenges. This internship has not only strengthened my cybersecurity competences, but has also increased my confidence in my own ability to adapt and thrive in new environments and with new technologies. I am grateful for this opportunity that has markedly enriched my professional and personal development.

Bibliographical References

[1] "Portal do Estudante — Politécnico da Guarda — politecnicoguarda.pt," https://politecnicoguarda.pt/portal-do-estudante/, [Accessed 30-06-2024].

[2] A. Jacinto, "Historia del aprendizaje automático: La cronología completa [ACTUALIZADO] — startechup.com," https://www.startechup.com/es/blog/machine-learning-history/4, [Accessed 07-02-2024].

[3] "1. Supervised learning — scikit-learn.org," https://scikit-learn.org/stable/supervised_learning.html, [Accessed 07-02-2024].

[4] "2. Unsupervised learning — scikit-learn.org," https://scikit-learn.org/stable/unsupervised_learning.html, [Accessed 14-02-2024].

[5] "What is Reinforcement Learning? – Overview of How it Works | Synopsys — synopsys.com," https://www.synopsys.com/ai/what-is-reinforcement-learning.html, [Accessed 22-02-2024].

[6] https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf, [Accessed 22-02-2024].

[7] "9 Reinforcement Learning Real-Life Applications — v7labs.com," https://www.v7labs.com/blog/reinforcement-learning-applications, [Accessed 20-03-2024].

[8] "What Is Deep Learning? | IBM — ibm.com," https://www.ibm.com/topics/deep-learning#:~:text=Deep%20learning%20is%20a%20subset,applications%20in%20our%20lives%20today., [Accessed 20-03-2024].

[9] mussem, "What is Deep Learning, Nature of Machine Learning and Beauty of Deep Neural Networks ? - New World : Artificial Intelligence — newworldai.com," https://www.newworldai.com/what-is-deep-learning-nature-of-machine-learning-and-beauty-of-deep-neural-networks/, [Accessed 13-03-2024].

[10] "Autoencoders -Machine Learning - GeeksforGeeks — geeksforgeeks.org," https://www.geeksforgeeks.org/auto-encoders/, [Accessed 28-02-2024].

[11] S. Flores, "Variational Autoencoders are Beautiful | Blogs — compthree.com," https://www.compthree.com/blog/autoencoder/, [Accessed 28-02-2024].

[12] "ML | Underfitting and Overfitting - GeeksforGeeks — geeksforgeeks.org," https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/, [Accessed 12-02-2024].

[13] "Overfitting and underfitting in machine learning | SuperAnnotate — superannotate.com," https://www.superannotate.com/blog/overfitting-and-underfitting-in-machine-learning, [Accessed 12-02-2024].

[14] "What is homomorphic encryption, and why isn't it mainstream? — keyfactor.com," https://www.keyfactor.com/blog/what-is-homomorphic-encryption/, [Accessed 16-04-2024].

[15] "Homomorphic Encryption: How It Works | Splunk — splunk.com," https://www.splunk.com/en_us/blog/learn/homomorphic-encryption.html, [Accessed 16-04-2024].

[16] "Homomorphic encryption - Wikipedia — en.wikipedia.org," https://en.wikipedia.org/wiki/Homomorphic_encryption, [Accessed 20-06-2024].

[17] V. Kumar, "HETAL: New Privacy-Preserving Method for Transfer Learning with Homomorphic Encryption — marktechpost.com," https://www.marktechpost.com/2024/03/27/hetal-new-privacy-preserving-method-for-transfer-learning-with-homomorphic-encryption/, [Accessed 16-04-2024].

[18] "What is Differential Privacy: definition, mechanisms, and examples - Statice — statice.ai," https://www.statice.ai/post/what-is-differential-privacy-definition-mechanisms-examples#:~:text=Differential%20privacy%20is%20a%20mathematical,any%20individual%20in%20the%20dataset., [Accessed 23-04-2024].

[19] J. Arenas, "'Differential privacy': el sistema clave para garantizar la privacidad de datos — redseguridad.com," https://www.redseguridad.com/especialidades-tic/proteccion-de-datos/differential-privacy-el-sistema-clave-para-garantizar-la-privacidad-de-los-datos_20210222.html, [Accessed 23-04-2024].

[20] "Zero-Knowledge Proof (ZKP) — Explained | Chainlink — chain.link," https://chain.link/education/zero-knowledge-proof-zkp, [Accessed 20-06-2024].

[21] "Zero-knowledge proofs explained in 3 examples — circularise.com," https://www.circularise.com/blogs/zero-knowledge-proofs-explained-in-3-examples, [Accessed 08-05-2024].

[22] "What Is Multiparty Computation? - IEEE Digital Privacy — digitalprivacy.ieee.org," https://digitalprivacy.ieee.org/publications/topics/what-is-multiparty-computation#:~:text=At%20its%20heart%2C%20multiparty%20computation,party%20learns%20anything%20about%20others., [Accessed 08-05-2024].

[23] R. Tosirisuk, "Overview, Applications, and Use Cases

of Multi-Party Computation (MPC) for Key Management — medium.com," https://medium.com/finema/overview-applications-and-use-cases-of-multiparty-computation-mpc-for-key-management-5 [Accessed 09-05-2024].

[24] IUDX, "Secure Multi-Party Computation Solutions | IUDX — iudx.org.in," https://iudx.org.in/secure-multi-party-computation/, [Accessed 10-05-2024].

[25] S. Bag, "A Beginners Guide to Federated Learning — analyticsvidhya.com," https://www.analyticsvidhya.com/blog/2021/05/federated-learning-a-beginners-guide/, [Accessed 06-02-2024].

[26] "Aprendizaje federado | TensorFlow Federated — tensorflow.org," https://www.tensorflow.org/federated/federated_learning?hl=es-419, [Accessed 07-02-2024].

[27] "A Step-by-Step Guide to Federated Learning in Computer Vision — v7labs.com," https://www.v7labs.com/blog/federated-learning-guide, [Accessed 06-02-2024].

[28] "Visual Studio Code - Wikipedia, la enciclopedia libre — es.wikipedia.org," https://es.wikipedia.org/wiki/Visual_Studio_Code, [Accessed 11-06-2024].

[29] "Visual Studio: IDE y Editor de código para desarrolladores de software y Teams — visualstudio.microsoft.com," https://visualstudio.microsoft.com/es/, [Accessed 11-06-2024].

[30] "Python - Wikipedia, la enciclopedia libre — es.wikipedia.org," https://es.wikipedia.org/wiki/Python, [Accessed 12-06-2024].

[31] "El tutorial de Python — docs.python.org," https://docs.python.org/es/3/tutorial/, [Accessed 12-06-2024].

[32] "TensorFlow — tensorflow.org," https://www.tensorflow.org/?hl=es-419, [Accessed 26-03-2024].

[33] T. F. Authors, "Flower: A Friendly Federated Learning Framework — flower.ai," https://flower.ai/, [Accessed 26-03-2024].

[34] "Home — docs.docker.com," https://docs.docker.com/, [Accessed 20-06-2024].

[35] "Orquestación de contenedores para producción — kubernetes.io," https://kubernetes.io/es/, [Accessed 20-06-2024].

[36] "MQTT - The Standard for IoT Messaging — mqtt.org," https://mqtt.org/, [Accessed 20-06-2024].

[37] "Eclipse Mosquitto — mosquitto.org," https://mosquitto.org/, [Accessed 20-06-2024].

[38] "mnista | TensorFlow Datasets — tensorflow.org," https://www.tensorflow.org/datasets/catalog/mnist?hl=es, [Accessed 05-07-2024].

[39] “noknow — pypi.org,” https://pypi.org/project/noknow/, [Accessed 21-06-2024].

[40] “noknow-python/README.md at master · GoodiesHQ/noknow-python — github.com,” https://github.com/GoodiesHQ/noknow-python/blob/master/README.md, [Accessed 21-06-2024].

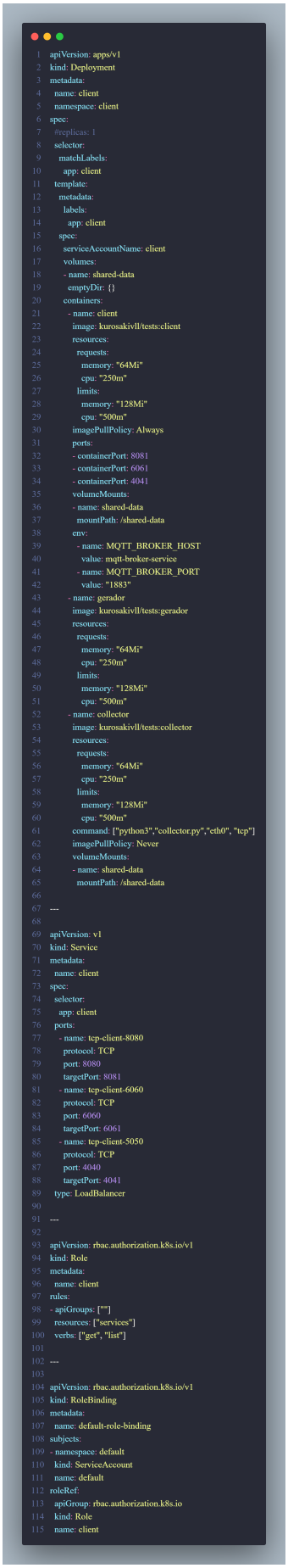
Appendix

```
1  FROM python:3.11-slim AS build
2
3  USER root
4
5  WORKDIR /app/
6
7  RUN apt-get update && apt-get install -y sudo
8
9  # Copiar todo o que esta dentro da pasta
10 COPY . .
11 RUN pip install --user --no-cache-dir --no-warn-script-location -r requirements.txt
12 #RUN pip install -r requirements.txt
13
14 # Exponer el puerto
15 EXPOSE 8080
16
17 # Comando para ejecutar el servidor
18 CMD ["python", "main.py"]
```

Figure 5.1: Dockerfile



Figure 5.2: Requirements



```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client
  namespace: client
spec:
  #replicas: 1
  selector:
    matchLabels:
      app: client
  template:
    metadata:
      labels:
        app: client
    spec:
      serviceAccountName: client
      volumes:
      - name: shared-data
        emptyDir: {}
      containers:
        - name: client
          image: kurosakivll/tests:client
          resources:
            requests:
              memory: "64Mi"
              cpu: "250m"
            limits:
              memory: "128Mi"
              cpu: "500m"
          imagePullPolicy: Always
          ports:
          - containerPort: 8081
          - containerPort: 6061
          - containerPort: 4041
          volumeMounts:
          - name: shared-data
            mountPath: /shared-data
          env:
            - name: MQTT_BROKER_HOST
              value: mqtt-broker-service
            - name: MQTT_BROKER_PORT
              value: "1883"
        - name: gerador
          image: kurosakivll/tests:gerador
          resources:
            requests:
              memory: "64Mi"
              cpu: "250m"
            limits:
              memory: "128Mi"
              cpu: "500m"
        - name: collector
          image: kurosakivll/tests:collector
          resources:
            requests:
              memory: "64Mi"
              cpu: "250m"
            limits:
              memory: "128Mi"
              cpu: "500m"
          command: ["python3","collector.py","eth0", "tcp"]
          imagePullPolicy: Never
          volumeMounts:
          - name: shared-data
            mountPath: /shared-data

---

apiVersion: v1
kind: Service
metadata:
  name: client
spec:
  selector:
    app: client
  ports:
    - name: tcp-client-8080
      protocol: TCP
      port: 8080
      targetPort: 8081
    - name: tcp-client-6060
      protocol: TCP
      port: 6060
      targetPort: 6061
    - name: tcp-client-5050
      protocol: TCP
      port: 4040
      targetPort: 4041
  type: LoadBalancer

---

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: client
rules:
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list"]

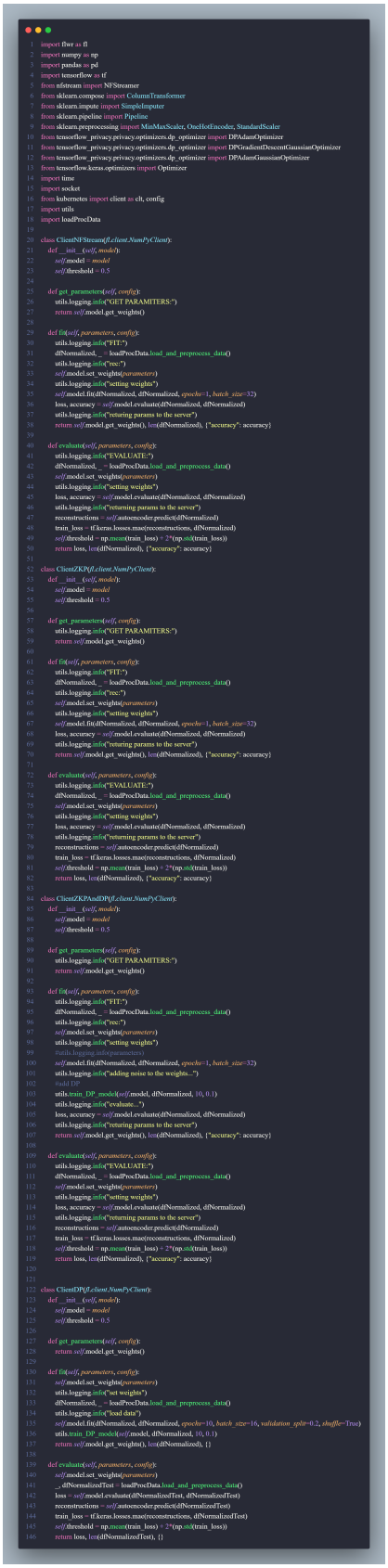
---

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: default-role-binding
subjects:
- namespace: default
  kind: ServiceAccount
  name: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: client
```

Figure 5.3: .yaml



Figure 5.4: main.py Client



```python
import flwr as fl
import numpy as np
import pandas as pd
import tensorflow as tf
from nfstream import NFStreamer
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder, StandardScaler
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPAdamOptimizer
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPGradientDescentGaussianOptimizer
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPAdamGaussianOptimizer
from tensorflow.keras.optimizers import Optimizer
import time
import socket
from kubernetes import client as clt, config
import utils
import loadProcData

class ClientNFStream(fl.client.NumPyClient):
    def __init__(self, model):
        self.model = model
        self.threshold = 0.5

    def get_parameters(self, config):
        utils.logging.info("GET PARAMITERS:")
        return self.model.get_weights()

    def fit(self, parameters, config):
        utils.logging.info("FIT:")
        dfNormalized, _ = loadProcData.load_and_preprocess_data()
        utils.logging.info("rec:")
        self.model.set_weights(parameters)
        utils.logging.info("setting weights")
        self.model.fit(dfNormalized, dfNormalized, epochs=1, batch_size=32)
        loss, accuracy = self.model.evaluate(dfNormalized, dfNormalized)
        utils.logging.info("returing params to the server")
        return self.model.get_weights(), len(dfNormalized), {"accuracy": accuracy}

    def evaluate(self, parameters, config):
        utils.logging.info("EVALUATE:")
        dfNormalized, _ = loadProcData.load_and_preprocess_data()
        self.model.set_weights(parameters)
        utils.logging.info("setting weights")
        loss, accuracy = self.model.evaluate(dfNormalized, dfNormalized)
        utils.logging.info("returing params to the server")
        reconstructions = self.autoencoder.predict(dfNormalized)
        train_loss = tf.keras.losses.mae(reconstructions, dfNormalized)
        self.threshold = np.mean(train_loss) + 2*(np.std(train_loss))
        return loss, len(dfNormalized), {"accuracy": accuracy}

class ClientZKP(fl.client.NumPyClient):
    def __init__(self, model):
        self.model = model
        self.threshold = 0.5

    def get_parameters(self, config):
        utils.logging.info("GET PARAMITERS:")
        return self.model.get_weights()

    def fit(self, parameters, config):
        utils.logging.info("FIT:")
        dfNormalized, _ = loadProcData.load_and_preprocess_data()
        utils.logging.info("rec:")
        self.model.set_weights(parameters)
        utils.logging.info("setting weights")
        self.model.fit(dfNormalized, dfNormalized, epochs=1, batch_size=32)
        loss, accuracy = self.model.evaluate(dfNormalized, dfNormalized)
        utils.logging.info("returing params to the server")
        return self.model.get_weights(), len(dfNormalized), {"accuracy": accuracy}

    def evaluate(self, parameters, config):
        utils.logging.info("EVALUATE:")
        dfNormalized, _ = loadProcData.load_and_preprocess_data()
        self.model.set_weights(parameters)
        utils.logging.info("setting weights")
        loss, accuracy = self.model.evaluate(dfNormalized, dfNormalized)
        utils.logging.info("returing params to the server")
        reconstructions = self.autoencoder.predict(dfNormalized)
        train_loss = tf.keras.losses.mae(reconstructions, dfNormalized)
        self.threshold = np.mean(train_loss) + 2*(np.std(train_loss))
        return loss, len(dfNormalized), {"accuracy": accuracy}

class ClientZKPAndDP(fl.client.NumPyClient):
    def __init__(self, model):
        self.model = model
        self.threshold = 0.5

    def get_parameters(self, config):
        utils.logging.info("GET PARAMITERS:")
        return self.model.get_weights()

    def fit(self, parameters, config):
        utils.logging.info("FIT:")
        dfNormalized, _ = loadProcData.load_and_preprocess_data()
        utils.logging.info("rec:")
        self.model.set_weights(parameters)
        utils.logging.info("setting weights")
        #train_loader = torch.utils.data.DataLoader(
        self.model.fit(dfNormalized, dfNormalized, epochs=1, batch_size=32)
        utils.logging.info("adding noise to the weights...")
        #add DP
        utils.train_DP_model(self.model, dfNormalized, 10, 0.1)
        utils.logging.info("evaluate...")
        loss, accuracy = self.model.evaluate(dfNormalized, dfNormalized)
        utils.logging.info("returing params to the server")
        return self.model.get_weights(), len(dfNormalized), {"accuracy": accuracy}

    def evaluate(self, parameters, config):
        utils.logging.info("EVALUATE:")
        dfNormalized, _ = loadProcData.load_and_preprocess_data()
        self.model.set_weights(parameters)
        utils.logging.info("setting weights")
        loss, accuracy = self.model.evaluate(dfNormalized, dfNormalized)
        utils.logging.info("returing params to the server")
        reconstructions = self.autoencoder.predict(dfNormalized)
        train_loss = tf.keras.losses.mae(reconstructions, dfNormalized)
        self.threshold = np.mean(train_loss) + 2*(np.std(train_loss))
        return loss, len(dfNormalized), {"accuracy": accuracy}


class ClientDP(fl.client.NumPyClient):
    def __init__(self, model):
        self.model = model
        self.threshold = 0.5

    def get_parameters(self, config):
        return self.model.get_weights()

    def fit(self, parameters, config):
        self.model.set_weights(parameters)
        utils.logging.info("set weights")
        dfNormalized, _ = loadProcData.load_and_preprocess_data()
        utils.logging.info("load data")
        self.model.fit(dfNormalized, dfNormalized, epochs=10, batch_size=16, validation_split=0.2, shuffle=True)
        utils.train_DP_model(self.model, dfNormalized, 10, 0.1)
        return self.model.get_weights(), len(dfNormalized), {}

    def evaluate(self, parameters, config):
        self.model.set_weights(parameters)
        _, dfNormalizedTest = loadProcData.load_and_preprocess_data()
        loss = self.model.evaluate(dfNormalizedTest, dfNormalizedTest)
        reconstructions = self.autoencoder.predict(dfNormalizedTest)
        train_loss = tf.keras.losses.mae(reconstructions, dfNormalizedTest)
        self.threshold = np.mean(train_loss) + 2*(np.std(train_loss))
        return loss, len(dfNormalizedTest), {}
```

Figure 5.5: ClassClient.py

```python
import flwr as fl
import numpy as np
import pandas as pd
import tensorflow as tf
from nfstream import NFStreamer
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder, StandardScaler
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPAdamOptimizer
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPGradientDescentGaussianOptimizer
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPAdamGaussianOptimizer
from tensorflow.keras.optimizers import Optimizer
import time
import socket
from kubernetes import client as clt, config
import loadProcData
import logging

def training_values():
    # Definires os dados de treino e proba
    (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
    x_train, x_test = x_train / 255.0, x_test / 255.0
    return x_train, y_train, x_test, y_test

def local_evaluate(model):
    try:
        inputTest = loadProcData.load_and_preprocess_data()
        reconstructed = model.predict(inputTest)
        mae = np.mean(np.abs(inputTest - reconstructed), axis=1)

        predictions = (mae > 0.5).astype(int)

        rowsToDrop = [i for i,val in enumerate(predictions) if val != 0]
        if len(rowsToDrop) > 0 :
            df = pd.read_csv('/shared-data/out.csv')
            for id, row in enumerate(rowsToDrop):
                logging.info("POTENTIAL ATTACK DETECTED!")
                logging.info("Removing it from the CSV")
                df.drop(index=row, inplace=True)
            df.to_csv('/shared-data/out.csv', index=False, mode='w')
        else:
            logging.info("No threats detected")
    except Exception as e:
        print("Error while trying to evaluate: " + str(e))

def create_model(dfNormalized):
    # Definir el autoencoder
    input_size = dfNormalized.shape[1]
    autoencoder = tf.keras.Sequential([
        tf.keras.layers.Dense(input_size, activation='relu'),
        tf.keras.layers.Dense(34, activation='relu'),
        tf.keras.layers.Dense(17, activation='relu'),
        tf.keras.layers.Dense(10, activation='relu'),
        tf.keras.layers.Dense(17, activation='relu'),
        tf.keras.layers.Dense(34, activation='relu'),
        tf.keras.layers.Dense(input_size, activation='sigmoid')
    ])

    # Compilar el modelo
    autoencoder.compile(optimizer=tf.keras.optimizers.Adam(),
                        loss=tf.keras.losses.MeanSquaredError())

    autoencoder.fit(dfNormalized, dfNormalized, epochs=10, batch_size=16)
    return autoencoder

def add_noise_to_weights(model, noise_multiplier):
    for layer in model.layers:
        for param in layer.trainable_weights:
            param.assign_add(tf.random.normal(tf.shape(param), stddev=noise_multiplier))

def train_DP_model(model, X, num_epochs=10, noise_multiplier=0.1):
    for epoch in range(num_epochs):
        add_noise_to_weights(model, noise_multiplier)

    return model
```

Figure 5.6: utils.py Client



```python
import flwr as fl
import numpy as np
import pandas as pd
import tensorflow as tf
from nfstream import NFStreamer
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder, StandardScaler
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPAdamOptimizer
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPGradientDescentGaussianOptimizer
from tensorflow_privacy.privacy.optimizers.dp_optimizer import DPAdamGaussianOptimizer
from tensorflow.keras.optimizers import Optimizer
import time
import socket
from kubernetes import client as clt, config
import utils

def load_and_preprocess_data():
    # Cargar datos desde un archivo CSV y convertirlo en un DataFrame de pandas.
    df = pd.DataFrame(pd.read_csv('/shared-data/out.csv'))
    dfTest = pd.DataFrame(pd.read_csv('./shared-data/out.csv'))

    # Comprobar si la columna 'Label' está presente en el DataFrame
    if 'Label' in df.columns:
        # Dividir características y etiquetas
        X = df.drop(columns=['Label'])
    else:
        # Si la columna 'Label' no está presente, utilizar solo las características X
        X = df

    # Definir las columnas numéricas y categóricas
    numeric_features = X.select_dtypes(include=['float64', 'int64']).columns
    categorical_features = X.select_dtypes(include=['object']).columns

    # Construir un preprocesador para las características numéricas
    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='mean')),
        ('scaler', StandardScaler())])

    # Construir un preprocesador para las características categóricas
    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))])

    # Combinar preprocesadores para todas las características
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', numeric_transformer, numeric_features),
            ('cat', categorical_transformer, categorical_features)])

    columns_to_drop = ['id', 'expiration_id', 'src_ip', 'src_mac', 'src_oui', 'src_port', 'dst_ip', 'dst_mac',
                       'dst_oui']
    df.drop(columns=columns_to_drop, inplace=True)
    dfTest.drop(columns=columns_to_drop, inplace=True)

    imputer = SimpleImputer(strategy='mean')
    dfImputed = imputer.fit_transform(df)
    dfTestImputed = imputer.fit_transform(dfTest)

    scaler = MinMaxScaler()
    dfNormalized = scaler.fit_transform(dfImputed)
    dfNormalizedTest = scaler.fit_transform(dfTestImputed)
    return dfNormalized, dfNormalizedTest
```

Figure 5.7: LoadProcData.py Client



```python
import flwr as fl
import tensorflow as tf
import logging
import socket
import schedule
import time
from getpass import getpass
from tinyhow.core import ZK, ZKSignature, ZKParameters, ZKData, ZKProof
from queue import Queue
from threading import Thread
from multiprocessing import Process
import pickle
from secrets import token_hex
from enum import Enum
import SqLiteDB
import json
import paho.mqtt.client as mqtt

DATABASE_NAME = "database.db"

class ClientTypes(Enum):
    NONE = 1
    ZKP = 2
    DP = 3
    BOTH = 4

def select_client_type():
    print("Select client type:")
    print("1. None")
    print("2. ZKP")
    print("3. DP")
    print("4. BOTH")
    choice = int(input("Enter the number of the client type: ")) # pasar a variable ambiente
    return ClientTypes(choice)

def define_strategy():
    return fl.server.strategy.FedAvg(min_available_clients=1, min_fit_clients=1, min_evaluate_clients=1)

def start_federated_round():
    connect_process = Process(target=start_flower_server)
    connect_process.start()

def start_flower_server():
    try:
        ip_address = socket.gethostbyname(socket.gethostname())
        server_address= ip_address + ':8080'
        logging.info(server_address)

        # Start Flower Server
        fl.server.start_server(
            server_address= server_address,
            config=fl.server.ServerConfig(num_rounds=1),
            strategy = define_strategy()
        )

    except Exception as e:
        logging.error("Exception while receiving the clients: " + str(e))

def wait_option(flag_option):
    logging.info("waiting for clients...")
    # Receive proof from client
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
            server_socket.bind(('0.0.0.0', 4040))
            server_socket.listen()
            while True:
                connection, address = server_socket.accept()
                logging.info("CLIENT CONNECTED")
                process = Process(target=handle_client_option, args=(connection, flag_option))
                process.start()
    except Exception as e:
        logging.error("Error in serverZKP: " + str(e))

def handle_client_option(connection, flag_option):
    with connection:
        try:
            logging.info("Sending client type: {ClientType(flag_option).name} ")
            connection.send(pickle.dumps(flag_option.value))
        except Exception as e:
            logging.error("Error handle client option: " + str(e))

def serverZKP(flag_option):
    logging.info("FUNCTION SEVERZKP")
    # Receive proof from client
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
            server_socket.bind(('0.0.0.0', 6060))
            server_socket.listen()
            while True:
                connection, address = server_socket.accept()
                logging.info("CLIENT CONNECTED")
                process = Process(target=handle_client, args=(connection,flag_option))
                process.start()
    except Exception as e:
        logging.error("Error in serverZKP: " + str(e))

# Función para manejar la conexión de un cliente
def handle_client(connection, flag_option):
    with connection:
        try:
            # Set up server component
            logging.info("generating key server...")
            password = token_hex(16)
            server_zk = ZK.new(curve_name="secp384r1", hash_alg="sha3_512")
            server_signature: ZKSignature = server_zk.create_signature(password)
            # Load the received signature from the Client
            logging.info("receiving signature...")
            sig = pickle.loads(connection.recv(1048576))
            logging.info("deserializing signature...")
            client_signature = ZKSignature.load(sig)
            logging.info("creating zk client...")
            client_zk = ZK(client_signature.params)
            # Create a signed token and send to the client
            logging.info("creating token...")
            token = server_zk.sign(password, client_zk.token())
            logging.info("sending token...")
            #connection.send(token.dumps(separator=":").encode())
            connection.send(pickle.dumps(token.dump(separator=":")))
            # Get the token from the client
            logging.info("getting proof...")
            proof = ZKData.load(pickle.loads(connection.recv(1048576)))
            logging.info("getting proof...")
            token = ZKData.load(proof.data, ":")
            # if the server signs the token so it can be sure it has not been modified
            if not server_zk.verify(token, server_signature):
                logging.info("aborting...")
                connection.send(pickle.dumps("False"))
            else:
                logging.info("verifying.....")
                if client_zk.verify(proof, client_signature, data=token):
                    connection.send(pickle.dumps("True"))
                    logging.info("sending flag...")
                    connection.send(pickle.dumps(flag_option.value))
                else:
                    connection.send(pickle.dumps("False"))
        except Exception as e:
            logging.error("Error while processing client data: " + str(e))
            connection.send(pickle.dumps("False"))

def main():
    logging.basicConfig(level=logging.INFO)
    SqLiteDB.create_db()
    flag_option = select_client_type()

    server_option_process = Process(target= wait_option, args= (flag_option,))
    server_option_process.start()
    server_zkp_process = Process(target=serverZKP, args=(flag_option,))
    server_zkp_process.start()

    # Configurar el cliente MQTT
    mqtt_client = mqtt.Client()
    mqtt_client.on_connect = SqLiteDB.on_connect
    mqtt_client.on_message = SqLiteDB.on_message
    mqtt_client.connect("emqx-deployment", 1883, 60)
    mqtt_client.loop_start()

    start_federated_round()
    schedule.every(5).minutes.do(start_federated_round)

    while True:
        schedule.run_pending()
        time.sleep(1)

if __name__ == "__main__":
    main()
```

Figure 5.8: main.py Server

```python
import flwr as fl
import tensorflow as tf
import paho.mqtt.client as mqtt
import json
import sqlite3
import logging

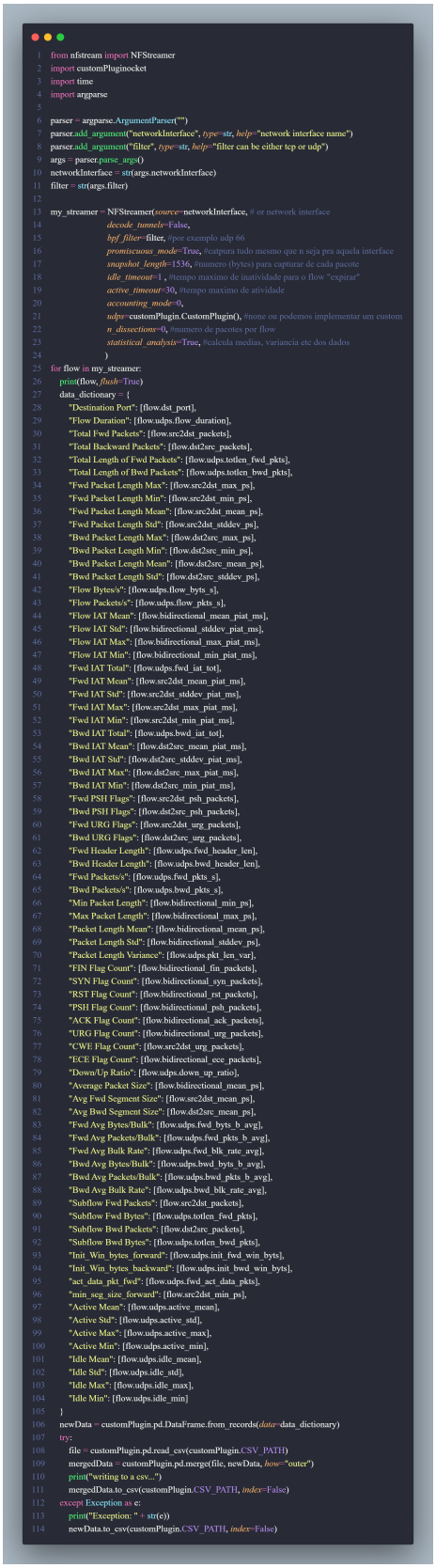
# Crear la tabla en la base de datos SQLite para almacenar los pesos del modelo
def createDb():
    connection = sqlite3.connect(DATABASE_NAME)
    cur = connection.cursor()
    cur.execute('''CREATE TABLE IF NOT EXISTS client_metrics (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    weights TEXT,
                    threshold REAL,
                    current REAL,
                    peak REAL,
                    time REAL
                )''')
    connection.commit()
    connection.close()

# Insertar los pesos del modelo en la base de datos SQLite
def insert_model_weights(weights, threshold, current, peak, time_spent):
    conn = sqlite3.connect('model_weights.db')
    c = conn.cursor()
    c.execute("INSERT INTO model_weights (weights, threshold, current, peak, time_spent) VALUES (?,?,?,?,?)",
              (json.dumps(weights), threshold, current, peak, time_spent))
    conn.commit()
    conn.close()

# MQTT callback functions
def on_connect(client, userdata, flags, rc):
    logging.info(f"Connected with result code {rc}")
    client.subscribe("receive parameters")

def on_message (client, userdata, msg):
    try:
        data = json.loads(msg.payload)
        insert_model_weights(data['parameters'], data['threshold'], data['current'], data['peak'], data['time'])
        logging.info("Data inserted into the database successfully.")
    except Exception as e:
        logging.info(f"Failed to insert data: {str(e)}")
```

Figure 5.9: SqLite.py Server



```python
from nfstream import NFStreamer
import customPluginocket
import time
import argparse

parser = argparse.ArgumentParser("")
parser.add_argument("networkInterface", type=str, help="network interface name")
parser.add_argument("filter", type=str, help="filter can be either tcp or udp")
args = parser.parse_args()
networkInterface = str(args.networkInterface)
filter = str(args.filter)

my_streamer = NFStreamer(source=networkInterface, # or network interface
                         decode_tunnels=False,
                         bpf_filter=filter, #por exemplo udp 66
                         promiscuous_mode=True, #captura tudo mesmo que n seja pra aquela interface
                         snapshot_length=1536, #numero (bytes) para capturar de cada pacote
                         idle_timeout=1 , #tempo maximo de inatividade para o flow "expirar"
                         active_timeout=30, #tempo maximo de atividade
                         accounting_mode=0,
                         udps=customPlugin.CustomPlugin(), #none ou podemos implementar um custom
                         n_dissections=0, #numero de pacotes por flow
                         statistical_analysis=True, #calcula medias, variancia etc dos dados
                         )
for flow in my_streamer:
    print(flow, flush=True)
    data_dictionary = {
        "Destination Port": [flow.dst_port],
        "Flow Duration": [flow.udps.flow_duration],
        "Total Fwd Packets": [flow.src2dst_packets],
        "Total Backward Packets": [flow.dst2src_packets],
        "Total Length of Fwd Packets": [flow.udps.totlen_fwd_pkts],
        "Total Length of Bwd Packets": [flow.udps.totlen_bwd_pkts],
        "Fwd Packet Length Max": [flow.src2dst_max_ps],
        "Fwd Packet Length Min": [flow.src2dst_min_ps],
        "Fwd Packet Length Mean": [flow.src2dst_mean_ps],
        "Fwd Packet Length Std": [flow.src2dst_stddev_ps],
        "Bwd Packet Length Max": [flow.dst2src_max_ps],
        "Bwd Packet Length Min": [flow.dst2src_min_ps],
        "Bwd Packet Length Mean": [flow.dst2src_mean_ps],
        "Bwd Packet Length Std": [flow.dst2src_stddev_ps],
        "Flow Bytes/s": [flow.udps.flow_byts_s],
        "Flow Packets/s": [flow.udps.flow_pkts_s],
        "Flow IAT Mean": [flow.bidirectional_mean_piat_ms],
        "Flow IAT Std": [flow.bidirectional_stddev_piat_ms],
        "Flow IAT Max": [flow.bidirectional_max_piat_ms],
        "Flow IAT Min": [flow.bidirectional_min_piat_ms],
        "Fwd IAT Total": [flow.udps.fwd_iat_tot],
        "Fwd IAT Mean": [flow.src2dst_mean_piat_ms],
        "Fwd IAT Std": [flow.src2dst_stddev_piat_ms],
        "Fwd IAT Max": [flow.src2dst_max_piat_ms],
        "Fwd IAT Min": [flow.src2dst_min_piat_ms],
        "Bwd IAT Total": [flow.udps.bwd_iat_tot],
        "Bwd IAT Mean": [flow.dst2src_mean_piat_ms],
        "Bwd IAT Std": [flow.dst2src_stddev_piat_ms],
        "Bwd IAT Max": [flow.dst2src_max_piat_ms],
        "Bwd IAT Min": [flow.dst2src_min_piat_ms],
        "Fwd PSH Flags": [flow.src2dst_psh_packets],
        "Bwd PSH Flags": [flow.dst2src_psh_packets],
        "Fwd URG Flags": [flow.src2dst_urg_packets],
        "Bwd URG Flags": [flow.dst2src_urg_packets],
        "Fwd Header Length": [flow.udps.fwd_header_len],
        "Bwd Header Length": [flow.udps.bwd_header_len],
        "Fwd Packets/s": [flow.udps.fwd_pkts_s],
        "Bwd Packets/s": [flow.udps.bwd_pkts_s],
        "Min Packet Length": [flow.bidirectional_min_ps],
        "Max Packet Length": [flow.bidirectional_max_ps],
        "Packet Length Mean": [flow.bidirectional_mean_ps],
        "Packet Length Std": [flow.bidirectional_stddev_ps],
        "Packet Length Variance": [flow.udps.pkt_len_var],
        "FIN Flag Count": [flow.bidirectional_fin_packets],
        "SYN Flag Count": [flow.bidirectional_syn_packets],
        "RST Flag Count": [flow.bidirectional_rst_packets],
        "PSH Flag Count": [flow.bidirectional_psh_packets],
        "ACK Flag Count": [flow.bidirectional_ack_packets],
        "URG Flag Count": [flow.bidirectional_urg_packets],
        "CWE Flag Count": [flow.src2dst_urg_packets],
        "ECE Flag Count": [flow.bidirectional_ece_packets],
        "Down/Up Ratio": [flow.udps.down_up_ratio],
        "Average Packet Size": [flow.bidirectional_mean_ps],
        "Avg Fwd Segment Size": [flow.src2dst_mean_ps],
        "Avg Bwd Segment Size": [flow.dst2src_mean_ps],
        "Fwd Avg Bytes/Bulk": [flow.udps.fwd_byts_b_avg],
        "Fwd Avg Packets/Bulk": [flow.udps.fwd_pkts_b_avg],
        "Fwd Avg Bulk Rate": [flow.udps.fwd_blk_rate_avg],
        "Bwd Avg Bytes/Bulk": [flow.udps.bwd_byts_b_avg],
        "Bwd Avg Packets/Bulk": [flow.udps.bwd_pkts_b_avg],
        "Bwd Avg Bulk Rate": [flow.udps.bwd_blk_rate_avg],
        "Subflow Fwd Packets": [flow.src2dst_packets],
        "Subflow Fwd Bytes": [flow.udps.totlen_fwd_pkts],
        "Subflow Bwd Packets": [flow.dst2src_packets],
        "Subflow Bwd Bytes": [flow.udps.totlen_bwd_pkts],
        "Init_Win_bytes_forward": [flow.udps.init_fwd_win_byts],
        "Init_Win_bytes_backward": [flow.udps.init_bwd_win_byts],
        "act_data_pkt_fwd": [flow.udps.fwd_act_data_pkts],
        "min_seg_size_forward": [flow.src2dst_min_ps],
        "Active Mean": [flow.udps.active_mean],
        "Active Std": [flow.udps.active_std],
        "Active Max": [flow.udps.active_max],
        "Active Min": [flow.udps.active_min],
        "Idle Mean": [flow.udps.idle_mean],
        "Idle Std": [flow.udps.idle_std],
        "Idle Max": [flow.udps.idle_max],
        "Idle Min": [flow.udps.idle_min]
    }
    newData = customPlugin.pd.DataFrame.from_records(data=data_dictionary)
    try:
        file = customPlugin.pd.read_csv(customPlugin.CSV_PATH)
        mergedData = customPlugin.pd.merge(file, newData, how="outer")
        print("writing to a csv...")
        mergedData.to_csv(customPlugin.CSV_PATH, index=False)
    except Exception as e:
        print("Exception: " + str(e))
        newData.to_csv(customPlugin.CSV_PATH, index=False)
```

Figure 5.10: Collector



Figure 5.11: Plugin Collector



```python
import random
import time
import requests

def generateTcpData():
  websites = [
    "https://www.google.com/",
    "https://www.youtube.com/",
    "https://www.facebook.com/",
    "https://twitter.com/",
    "https://www.instagram.com/",
    "https://en.wikipedia.org/wiki/Main_Page",
    "https://www.amazon.com/",
    "https://www.reddit.com/",
    "https://www.netflix.com/",
    "https://www.linkedin.com/"
  ]

  while True:
    website = random.choice(websites)
    try:
      response = requests.get(website)
      print(response, flush=True)
    except Exception as e:
      print("error:" + str(e))
    time.sleep(random.uniform(1,3))

generateTcpData()
```

Figure 5.12: Generator